DOCUMENT TECHNIQUE

Convention de Codage — V2.1

Java 21 · Spring Boot 4.1 · PostgreSQL — TypeScript · Angular 21/22

VVS — Virtual Vision Synergy · Standard technique interne

Champ	Valeur
Version	2.1 — révision corrigée et complétée de la V2
Date	Octobre 2026
Portée	Tous projets VVS (dont Valimbavaka) — backend Java / Spring Boot, frontend Angular, base PostgreSQL, déploiement Docker
Architecture	FBL (Feature-Based Layering) par défaut ; Clean Architecture (Annexe E) pour les projets ou modules qui la justifient, décidée par ADR (Annexe D)
Audience	Développeurs VVS — débutant à senior, revue de code, onboarding
Sources	V2 VVS auditée et corrigée ; documentations officielles (Spring, Hibernate, PostgreSQL, Angular, OWASP, Docker) vérifiées en octobre 2026 — voir Annexe C
Statut	Applicable dès validation des décisions listées en Annexe B (réunion d'équipe)
Classification	Standard technique interne

Préambule

Ce document est le référentiel des normes de développement des projets VVS. Il couvre le backend (Java, Spring Boot, architecture Feature-Based Layering par défaut, Clean Architecture en option), le frontend (TypeScript, Angular), la base de données (PostgreSQL), la sécurité, les tests, la qualité automatisée, le workflow Git et le déploiement Docker.

Une règle de ce document est **obligatoire** sauf mention « recommandé ». Toute exception se justifie dans la PR (commentaire ou description) et, si elle devient récurrente, fait évoluer ce document par une PR dédiée.

Ce qui change en V2.1

- **Socle mis à jour** : Spring Boot 4.0 sort du support open source le 31/12/2026 — la cible devient Spring Boot 4.1.x. Angular 22 est sorti en juin 2026 ; Angular 21 reste supporté. Karma n'est plus le runner de tests par défaut (Vitest).
- **Contradictions internes corrigées** : format de réponse API vs types Angular, sérialisation de `Page`, `inject()` vs constructeur, seuils de duplication, code HTTP des erreurs métier, nommage DTO front/back, exceptions aux règles de nommage (tests, `.spec.ts`, `app.routes.ts`).
- **Exemples corrigés et cohérents entre eux** : login sans énumération par timing ni oubli du verrouillage, `security/` découplé des features, listeners d'événements après commit. Les extraits sont à valider dans le projet modèle (Annexe B.2).
- **Sections ajoutées** : socle et versions, conventions PostgreSQL, Java 21 moderne, argent et dates, OWASP Top 10:2025, formulaires Angular, qualité automatisée et CI, workflow Git, configuration / Docker / observabilité.
- **Une architecture par défaut, une alternative** : le backend suit le Feature-Based Layering (FBL) ; la Clean Architecture (Annexe E) est réservée aux projets ou modules dont l'ADR la justifie (Annexe D, avec questionnaire et modèle d'ADR). Tout le reste de la convention est commun.
- **Bonnes pratiques renforcées** : contrôleurs minces, pattern Result, Specification, règles async/timeouts, audit et soft delete, verrouillage de compte, contrat de pagination, grille de revue et checklist PR.
- **Annexe A** : journal de toutes les corrections. **Annexe B** : décisions que l'équipe doit trancher avant d'appliquer la convention.

Sommaire

0. Socle technique et versions
1. Conventions de nommage
2. Organisation et structure du code
3. Principes SOLID
4. Architecture Feature-Based Layering (FBL)
5. Design patterns
6. Clean Code Java
7. API REST
8. Spring Data JPA et PostgreSQL
9. Sécurité
10. Frontend Angular / TypeScript
11. Tests
12. Qualité automatisée et CI
13. Workflow Git et revue
14. Configuration, Docker et observabilité

15. Grille de revue de code
Annexe A — Journal des corrections V2 → V2.1
Annexe B — Décisions à valider
Annexe C — Références
Annexe D — Choisir l'architecture et faire évoluer le projet
Annexe E — Clean Architecture

0. Socle technique et versions

Les versions ci-dessous sont les cibles du projet. Elles sont figées dans `pom.xml` (ou `build.gradle`) et `package.json` ; toute montée de version majeure passe par une PR dédiée avec note de migration.

Composant	Cible	Remarques
Java	21 LTS (minimum)	Spring Boot 4 exige Java 17 au minimum et supporte officiellement Java 25 LTS. Passage à 25 : décision D1.
Spring Boot	4.1.x	Support open source de la 4.0 : jusqu'au 31/12/2026 ; de la 4.1 : jusqu'au 31/07/2027 (vérifié octobre 2026). Démarrer en 4.0 impose une seconde montée de version.
Spring Framework / Jakarta EE	7.x / EE 11	Gérés par le BOM Spring Boot — ne jamais forcer leur version à la main.
Jackson	3.x	Nouveau package `tools.jackson` (les annotations restent `com.fasterxml.jackson.annotation`). Jackson 2 n'est plus que déprécié.
Hibernate / Spring Data JPA	Versions du BOM Boot 4.1	Pas de surcharge de version.
PostgreSQL	Version majeure figée dans `docker-compose.yml`	Même version en dev, test (Testcontainers) et prod — décision D2.
Migrations	Flyway	Dépendance `flyway-database-postgresql` obligatoire avec PostgreSQL.
Angular	22 (nouveaux projets) ; 21 en LTS	v22 (3 juin 2026, support actif jusqu'en 06/2027, LTS 06/2028) : Signal Forms, `resource()`/`httpResource()` et `@angular/aria` stables, OnPush par défaut, backend HTTP `fetch`, `strictTemplates` par défaut. v21 (nov. 2025) est en LTS jusqu'en 06/2027 : zoneless et Vitest par défaut — décision D3.
TypeScript / Node.js	Angular 22 : TypeScript 6 requis, Node 20 abandonné	Node.js LTS supporté par la version Angular retenue ; fichier `.nvmrc` à la racine du front. Migrer 21 → 22 : schematics de la CLI (`ng update`), dont Karma → Vitest.
Conteneurs	Docker Engine (WSL2 en dev) + Compose	Images taguées par version, jamais `latest`.
Build backend	Maven ou Gradle	Un seul outil par dépôt, wrapper (`mvnw` / `gradlew`) versionné — décision D4.

> **Règle — Suivi des dépendances**
> Dependabot (ou Renovate) activé sur les deux dépôts, regroupement hebdomadaire des mises à jour mineures. Aucune dépendance ajoutée sans justification dans la PR (OWASP A03:2025 — chaîne d'approvisionnement logicielle).

1. Conventions de nommage

Le nommage est le premier indicateur de qualité d'un code source. Des noms cohérents et porteurs de sens rendent le code auto-documenté, limitent les commentaires et accélèrent les revues.

1.1 Backend — Java / Spring Boot

Élément	Convention	Exemple correct	Exemple incorrect
Classe	PascalCase	`OrderService`	`orderService`, `Order_Service`
Interface	PascalCase, sans préfixe I	`OrderService`	`IOrderService`
Implémentation	Nom + `Impl`, dans `service/impl/`	`OrderServiceImpl`	`OrderServiceImplementation`
Controller	Ressource + `Controller`	`OrderController`	`OrdersCtrl`
Méthode	camelCase, verbe	`calculateTotal()`	`CalculateTotal()`, `total()`
Variable locale / champ	camelCase, sans préfixe	`orderTotal`, `orderRepository`	`_orderRepository`, `mRepo`
Constante (`static final` immuable)	SCREAMING_SNAKE_CASE	`MAX_RETRY_COUNT`	`MaxRetryCount`
Logger	`private static final Logger log` (exception admise)	`log.info(...)`	`LOGGER`, `logger` par instance
Enum (type / valeur)	PascalCase singulier / SCREAMING_SNAKE	`OrderStatus.IN_PROGRESS`	`OrderStatuses.InProgress`
DTO (record)	`RDto` en sortie, `WDto` en entrée ; `Dto` seul si forme identique (référentiel simple)	`OrderRDto`, `CreateOrderWDto`, `HolidayDto`	`OrderDTO`, `OrderRequest`
Entité JPA	PascalCase singulier, dans `<feature>/domain/`	`Order`, `OrderLine`	`Orders`, `OrderEntity`
Mapper MapStruct	Entité + `Mapper` ; méthodes `toRDto`, `toEntity`, `updateEntity`	`OrderMapper.toRDto(order)`	`OrderConverter.convert()`
Événement	PascalCase au passé + `Event`	`OrderCreatedEvent`	`OnOrderCreated`, `CreateOrderEvent`
Propriétés de config	Préfixe + `Properties` (record)	`JwtProperties`	`JwtConfigValues`
Package	minuscules, singulier, sans séparateur	`com.vvs.stockmovement.domain`	`com.vvs.stock_movement`, `com.vvs.Orders`
Paramètre générique	Une majuscule (ou `ID`) — convention Java	`T`, `E`, `K`, `V`, `ID`	`TEntity`, `Type1`
Méthode de test	`methodName_scenario_expected` (underscore autorisé ici uniquement)	`createOrder_validData_returnsCreated`	`testCreateOrder1`

Correction V2.1

`TEntity` retiré : en Java, les paramètres génériques sont des majuscules uniques (`T`, `E`, `K`, `V`) ; Spring utilise `ID`.
Ajout de l'exception pour le logger (`log`) et pour les méthodes de test, qui contredisaient sinon les règles SCREAMING_SNAKE et « jamais de snake_case ».

1.2 Frontend — TypeScript / Angular

Règle d'alignement : **le suffixe de rôle présent dans le nom de fichier se retrouve dans le nom du symbole, et seulement lui.** Un composant `order-list.ts` exporte `OrderList` ; un service `order-service.ts` exporte `OrderService` ; un guard fonctionnel `auth-guard.ts` exporte `authGuard`.

Élément	Convention	Exemple correct	Exemple incorrect
Composant (fichiers)	kebab-case, sans suffixe	`order-list.ts / .html / .css`	`order-list.component.ts`
Composant (classe)	PascalCase, sans suffixe (aligné sur le fichier) — D5	`OrderList`	`OrderListComponent` avec un fichier sans suffixe
Service (fichier / classe)	kebab-case + `-service` / PascalCase + `Service`	`order-service.ts` → `OrderService`	`order.service.ts`, `order.ts` (collision avec le modèle `Order`)
Guard, interceptor, resolver, pipe, directive	kebab-case + `-guard`, `-interceptor`… (séparateur tiret)	`auth-guard.ts` → `authGuard`	`auth.guard.ts`, `AuthGuard.ts`
Fichiers à point conservés	`.spec.ts`, `*.routes.ts`, `app.config.ts`, `environment*.ts`	`order-list.spec.ts`, `order.routes.ts`	`order-list-spec.ts`
Type / interface DTO	Même nom que le DTO backend	`OrderRDto`, `CreateOrderWDto`	`IOrder`, `CreateOrderRequest`
Énumération métier	Union de littéraux ou objet `as const`, valeurs identiques au JSON backend	`type OrderStatus = 'DRAFT' | 'IN_PROGRESS'`	`enum OrderStatus { InProgress }` (valeur ≠ backend)
Variable, propriété, signal	camelCase, sans suffixe	`orderCount`, `isLoading`	`orderCountSignal`
Observable	camelCase + `$`	`orders$`	`ordersObservable`
Constante globale	SCREAMING_SNAKE_CASE	`MAX_PAGE_SIZE`	`maxPageSize`
Sélecteur composant	kebab-case + préfixe projet — D6	`vvs-order-list`	`orderList`
Dossier de feature	kebab-case	`stock-movement/`	`StockMovement/`
Test (spec)	`describe` = symbole, `it` = phrase lisible	`it('should display one row per order')`	`it('test1')`

> **Correction V2.1**
>
> La V2 nommait les DTO front `CreateOrderRequest` (§1.2) tout en exigeant un miroir strict du backend (§10.4) et en utilisant `CreateOrderWDto` (§10.2). Un seul nom désormais : celui du backend.
>
> Les enums TypeScript en `InProgress` ne correspondaient pas aux valeurs JSON `IN_PROGRESS` envoyées par Spring : les valeurs sont maintenant identiques au contrat API.
>
> La règle « jamais de point dans les noms de fichiers » cassait `*.spec.ts`, `app.routes.ts` et `app.config.ts` générés par la CLI : exceptions listées.
>
> Depuis Angular 20, la CLI génère les guards, interceptors et resolvers avec un tiret (`auth-guard.ts`). Les services sont générés sans suffixe (`order.ts`) : le suffixe `-service` reste un choix VVS, à fixer dans `angular.json` (schematics) pour que `ng generate` le respecte.

1.3 Nommage sémantique

Règle d'or : un nom révèle l'intention. Si un nom a besoin d'un commentaire pour être compris, le nom est mauvais.

✗ Noms obscurs

```
int d;                      // durée en jours ???
String s;                   // ???
var list1 = getData();      // quelles données ???
boolean flag;               // quel flag ???
void process();             // traiter quoi ???
```

✓ Noms expressifs

```
int elapsedDays;
String customerEmail;
Page<Order> activeOrders = orderRepository.findByStatus(OrderStatus.ACTIVE, pageable);
boolean isEligibleForDiscount;
void validatePayment(PaymentWDto payment);
const discountedTotal = calculateDiscountedTotal(order);
```

1.4 Interdits

- Abréviations, sauf universelles : `Id`, `Http`, `Url`, `Json`, `Xml`, `Sql`, `Dto`, `Uuid`.
- Noms à une lettre, sauf lambdas très courtes (`o -> o.getId()`) et paramètres génériques.
- Notation hongroise (`strName`, `bIsActive`) et préfixe `I` sur les interfaces.
- snake_case dans les noms publics Java/TS — **sauf** méthodes de test JUnit.
- Numérotation (`handler1`, `service3`) et mots génériques seuls (`Manager`, `Processor`, `Data`, `Info`, `Helper`, `Utils`) — une classe utilitaire porte un nom métier (`MoneyFormatter`, `DateRanges`).
- Négation dans les booléens (`isNotActive`) → `!isActive` ou `isInactive`.
- `any` en TypeScript sans `// eslint-disable-next-line` + justification.

2. Organisation et structure du code

2.1 Structure d'une classe Java

Ordre fixe : package, imports, déclaration, constantes, logger, champs injectés, constructeur, méthodes publiques, méthodes privées. Un fichier = un type public de haut niveau. L'ordre des imports n'est pas réglé à la main : il est appliqué par le formateur (§12).

Exemple de référence (service FBL)

```
package com.vvs.shop.feature.order.service.impl;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.vvs.shop.feature.order.domain.Order;
import com.vvs.shop.feature.order.dto.CreateOrderWDto;
import com.vvs.shop.feature.order.dto.OrderRDto;
import com.vvs.shop.feature.order.mapper.OrderMapper;
import com.vvs.shop.feature.order.repository.OrderRepository;
import com.vvs.shop.feature.order.service.OrderPricing;
import com.vvs.shop.feature.order.service.OrderService;

@Service
@Transactional(readOnly = true)              // lecture par défaut
public class OrderServiceImpl implements OrderService {

    private static final Logger log = LoggerFactory.getLogger(OrderServiceImpl.class);

    private final OrderRepository orderRepository;
    private final OrderMapper orderMapper;
    private final OrderPricing orderPricing;

    public OrderServiceImpl(OrderRepository orderRepository, OrderMapper orderMapper, OrderPricing
orderPricing) {
        this.orderRepository = orderRepository;
        this.orderMapper = orderMapper;
        this.orderPricing = orderPricing;
    }

    @Override
    @Transactional                           // écriture : surcharge explicite
    public OrderRDto create(CreateOrderWDto request) {
        var total = orderPricing.totalOf(request.lines());
        Order saved = orderRepository.save(Order.create(request.customerId(), total));
        log.info("Order {} created", saved.getId());
        return orderMapper.toRDto(saved);
    }
}
```

Correction V2.1

L'exemple V2 utilisait `OrderService`, `OrderMapper`, `OrderRDto` sans les importer et n'avait pas de logger alors que §6.7 l'impose.

`@Transactional(readOnly = true)` au niveau de la classe, `@Transactional` sur les écritures : on ne peut plus oublier la lecture seule (P1 en V2).

Décision à valider — Lombok (D7)

La V2 écrit les constructeurs à la main. Avec Lombok, `@RequiredArgsConstructor` et `@Getter` remplacent ce code ; `@Data`, `@EqualsAndHashCode` et `@ToString` restent interdits sur les entités JPA.

2.2 Limites de taille

Élément	Limite	Action si dépassé
Méthode / fonction	30 lignes	Extraire des méthodes privées
Classe / composant TS	300 lignes	Découper (SRP)
Fichier	1 type public	Nouveau fichier
Paramètres	4	Objet paramètre (record) — `Pageable` compte pour 1
Imbrication	3 niveaux	Early return, extraction
Ligne	120 caractères	Retour à la ligne (formateur)
Template Angular inline	15 lignes	`templateUrl` séparé
Héritage	2 niveaux	Composition

- **Imports** groupés dans cet ordre, une ligne vide entre groupes : `java.*` / `javax.*` → `jakarta.*` → `org.*` → autres bibliothèques → packages du projet (`com.vvs.*`). Aucun import inutile ni import `*` (Checkstyle `UnusedImports`, `AvoidStarImport`, bloquant en CI).
- **Avertissements de compilation** : 0. Le build échoue sur warning (`-Xlint:all -Werror` pour Maven/Gradle, `strict` côté TypeScript). Les exceptions personnalisées déclarent `serialVersionUID` (ou `-Xlint:-serial`), sinon `-Werror` échoue.
- Les packages suivent l'arborescence des dossiers ; le package racine est `com.vvs.<produit>` (`com.vvs.shop`) ; l'organisation interne dépend de l'architecture retenue (§2.3).

2.3 Backend — Feature-Based Layering

Le code est organisé par **feature métier**, en miroir du frontend. Chaque feature regroupe controller, service (interface + impl), domain, repository, dto, mapper et exception. Ce n'est pas du Domain-Driven Design : pas de couches domain / application / infrastructure séparées, ni d'agrégats formalisés. Un seul module Maven (ou Gradle), organisé en packages.

Arborescence

```
com.vvs.shop
├── ShopApplication.java
├── core                        // transverse technique, sans métier
│   ├── config                  // WebConfig, OpenApiConfig, JacksonConfig, ClockConfig
│   └── exception               // GlobalExceptionHandler, BusinessException, NotFoundException…
├── security                    // ne dépend d'AUCUNE feature
│   ├── config                  // SecurityConfig (SecurityFilterChain)
│   ├── jwt                     // JwtService, JwtProperties
│   ├── handler                 // RestAuthenticationEntryPoint, RestAccessDeniedHandler
│   └── annotation              // @HasPermission (méta-annotation @PreAuthorize)
├── shared                      // générique réutilisable, sans métier
│   ├── domain                  // BaseEntity (champs d'audit)
│   └── dto                     // PageResponse, PageQuery
└── feature
    ├── user                    // comptes et rôles ; implémente AuthenticatedUserLoader
    │   └── controller  service/impl  domain  repository  dto  mapper  exception
    ├── order                   // feature, même découpe
    └── stock                   // feature, même découpe
```

Correction V2.1

`security/middleware` renommé `security/jwt` + `security/handler` : « middleware » n'est pas un concept Spring, on parle de filtres et de handlers.

`security/manager/PermissionManager` supprimé : avec `@PreAuthorize` et les authorities du JWT, il doublonnait `@HasPermission` et utilisait un mot générique interdit en §1.4.

`core/util` remplacé par des packages nommés : la V2 interdisait « Utils » tout en créant un package `util`.

Règles feature-based

- Une feature = un contexte métier cohérent (`user`, `stock`, `catalog`, `sale`…), déplaçable sans casser les autres.
- `core/`, `security/` et `shared/` ne contiennent jamais de logique métier et ne dépendent d'aucune feature.
- Une feature n'importe jamais le `repository` ni le `mapper` d'une autre : elle passe par l'interface de service publique ou par un événement.
- **Relations JPA entre features** : référence par identifiant (`UUID customerId`), pas d'association `@ManyToOne` vers l'entité d'une autre feature — sauf dérogation listée dans l'ADR de la feature (D8).

2.4 Frontend — organisation par feature

Arborescence

```
src/app
├── app.config.ts  app.routes.ts  app.ts
├── core                          // singletons transverses
│   ├── http                      // auth-interceptor.ts, error-interceptor.ts, problem-detail.ts
│   ├── auth                      // auth-guard.ts, auth-service.ts, has-permission-directive.ts
│   └── layout                    // main-layout/
├── shared                        // composants, pipes et types génériques réutilisables
│   ├── components                // data-table/, confirm-dialog/
│   └── models                    // page-response.ts
└── features
    └── order                     // alignée sur la feature backend "order"
        ├── order.routes.ts       // chargée en lazy loading
        ├── data-access           // order-service.ts, order-models.ts (OrderRDto…)
        ├── pages                 // composants "smart" routés : order-list-page/
        └── components            // composants de présentation : order-table/, order-form/
```

- Composants standalone (comportement par défaut depuis Angular 19 : ne plus écrire `standalone: true`). Pas de NgModule, sauf librairie tierce qui l'exige.
- Chaque feature possède ses routes (`order.routes.ts`) chargées via `loadChildren` ; les pages via `loadComponent`.
- Les services d'accès aux données et les types d'une feature vivent **dans** la feature (`data-access/`). Seul le transverse va dans `core/` ou `shared/` — D9.
- Une feature n'importe jamais un fichier d'une autre feature ; ce qui est partagé migre dans `shared/` (règle ESLint de frontières, §12).

> **Correction V2.1**
> La V2 se disait feature-based mais plaçait tous les services et modèles dans des dossiers globaux `services/` et `models/`, et ne montrait aucun fichier de routes alors qu'elle imposait le lazy loading par feature.

2.5 Vérification automatique de l'architecture

Une règle d'architecture non testée finit toujours par être violée. Les règles de §2.3 et §4 sont vérifiées par des tests ArchUnit exécutés dans la CI (détail en §4.3).

3. Principes SOLID

Les cinq principes SOLID sont la base d'un code maintenable et testable, côté Java comme côté services et composants Angular.

3.1 Single Responsibility (SRP)

Une classe n'a qu'une raison de changer. Test simple : si sa description contient « et », elle en fait trop.

✓ **Responsabilités séparées**

```
OrderService            // orchestration métier des commandes
PasswordEncoder         // hachage (fourni par Spring Security)
EmailSender             // envoi d'e-mails
InvoicePdfGenerator     // génération de PDF
```

3.2 Open/Closed (OCP)

Ouvert à l'extension, fermé à la modification : un nouveau mode de livraison = une nouvelle classe, zéro modification du code existant.

```
public interface ShippingCalculator {
    ShippingMethod method();               // enum, pas une String libre
    BigDecimal calculate(Order order);
}

@Component
class StandardShipping implements ShippingCalculator { /* ... */ }

@Component
class ExpressShipping implements ShippingCalculator { /* ... */ }
// Spring injecte List<ShippingCalculator> : voir Strategy (§5.1)
```

3.3 Liskov Substitution (LSP)

- Un repository ou service qui lève `UnsupportedOperationException` sur une méthode héritée viole LSP.
- Une sous-classe ne durcit pas les préconditions et n'affaiblit pas les postconditions (pas de `null` là où le parent lève une exception).

3.4 Interface Segregation (ISP)

✓ **Interfaces ciblées**

```
public interface DocumentReader {
    DocumentRDto getById(UUID id);
    PageResponse<DocumentRDto> search(DocumentSearchQuery query, Pageable pageable);
}
public interface DocumentWriter   { DocumentRDto create(CreateDocumentWDto request); }
public interface DocumentWorkflow { void validate(UUID id); void reject(UUID id, String reason); }
public interface DocumentExporter { byte[] exportPdf(UUID id); }
```

> **Correction V2.1**
> L'exemple V2 renvoyait des entités (`Document`) et une `List` non paginée, et utilisait des paramètres à une lettre (`q`, `r`) : trois violations des règles du même document.

3.5 Dependency Inversion (DIP)

Les modules de haut niveau dépendent d'abstractions. En Spring : injection par constructeur unique (pas besoin de `@Autowired`), jamais sur un champ.

- Paramètres de constructeur attendus : interfaces (services, repositories, `PasswordEncoder`, `EmailSender`).

- Exceptions admises : records de configuration (`JwtProperties`), `Clock`, `JsonMapper`/`ObjectMapper`, mappers MapStruct, types valeur.
- Un `new` d'un service dans un constructeur est une violation (sauf factory dont c'est le rôle).

4. Architecture Feature-Based Layering (FBL)

Principe : on découpe d'abord par **fonctionnalité métier** (`order`, `customer`, `catalog`), puis, à l'intérieur de chaque feature, par **couche technique**. L'objectif est la lisibilité, des frontières nettes et un couplage faible entre features, avec un coût d'entrée faible.

4.1 Sens des dépendances

De ↓ / Vers →	controller	service	repository	domain	dto / mapper
controller	—	oui (interface)	non	non	oui
service (impl)	non	oui (autre feature : interface seule)	oui (la sienne)	oui	oui
repository	non	non	—	oui	non
domain	non	non	non	oui (même feature)	non
mapper	non	non	non	oui	oui

- Un controller n'appelle jamais un repository ni ne manipule une entité : il reçoit et renvoie des DTO.
- Une feature appelle une autre feature **uniquement via l'interface de service publique** (`CustomerLookup`), jamais via son repository ou son entité.
- Les dépendances cycliques entre features sont interdites ; en cas de besoin mutuel, extraire une interface ou publier un événement (§5.4).

4.2 Packages transverses

Package	Contenu	Peut dépendre des features ?
`core/`	Configuration (`@Configuration`, `@ConfigurationProperties`), exceptions de base, gestionnaire d'erreurs, horloge	Non
`security/`	Filtres, `SecurityFilterChain`, émission des jetons, `CurrentUser`	Non — il définit des ports, les features les implémentent
`shared/`	Types valeur génériques, `BaseEntity`, `PageResponse`, `PageQuery`	Non
`feature/*`	Code métier	Via interfaces uniquement

> **Correction V2.1 — Dépendance inversée dans l'exemple d'authentification**
>
> L'exemple V2 plaçait `security/` en dépendance directe de la feature `user` (repository + entité) : le transverse dépendait du métier.
>
> V2.1 : `security/` définit le **port** `AuthenticatedUserLoader`, et la feature `user` l'implémente. La vérification du mot de passe prend un temps constant même si l'utilisateur n'existe pas, et le verrouillage de compte est géré.

✓ **Authentification sans dépendance vers une feature**

```
// security/AuthenticatedUserLoader.java  (port, défini côté transverse)
public interface AuthenticatedUserLoader {
    Optional<AuthenticatedUser> findByEmail(String email);
    void recordFailure(UUID userId);
    void recordSuccess(UUID userId);
}

public record AuthenticatedUser(UUID id, String passwordHash, Set<String> permissions,
                                boolean enabled, Instant lockedUntil) {}

// security/AuthService.java
@Service
public class AuthService {

    private final AuthenticatedUserLoader users;
    private final PasswordEncoder encoder;
    private final TokenIssuer tokens;
    private final Clock clock;
    private final String dummyHash;   // hash factice calculé une fois avec le même encodeur (même
coût)

    public AuthService(AuthenticatedUserLoader users, PasswordEncoder encoder, TokenIssuer tokens,
Clock clock) {
        this.users = users; this.encoder = encoder; this.tokens = tokens; this.clock = clock;
        this.dummyHash = encoder.encode("not-a-real-password");
    }

    // Pas de @Transactional ici : l'échec doit rester enregistré alors qu'une exception est levée
    // (recordFailure / recordSuccess s'exécutent chacun dans leur propre transaction).
    public TokenPair login(String email, String rawPassword) {
        var user = users.findByEmail(email.strip().toLowerCase(Locale.ROOT));
        // Toujours comparer un hash : temps de réponse comparable si l'email est inconnu
        var hash = user.map(AuthenticatedUser::passwordHash).orElse(dummyHash);
        boolean matches = encoder.matches(rawPassword, hash);

        if (user.isEmpty() || !matches || !user.get().enabled() || isLocked(user.get())) {
            user.ifPresent(u -> users.recordFailure(u.id()));
            throw new BadCredentialsException("Identifiants invalides"); // message unique
        }
        users.recordSuccess(user.get().id());
        return tokens.issue(user.get());
    }

    private boolean isLocked(AuthenticatedUser u) {
        return u.lockedUntil() != null && u.lockedUntil().isAfter(clock.instant());
    }
}
```

4.3 Vérification automatique

Les règles de dépendance sont testées, pas seulement relues. Un test ArchUnit par dépôt (ou Spring Modulith, décision D10) fait échouer le build à la première violation.

✓ **Garde-fous ArchUnit**

```
@AnalyzeClasses(packagesOf = ShopApplication.class, importOptions =
ImportOption.DoNotIncludeTests.class)
class ArchitectureTest {

    @ArchTest
    static final ArchRule controllers_do_not_touch_repositories =
        noClasses().that().resideInAPackage("..controller..")
            .should().dependOnClassesThat().resideInAPackage("..repository..");

    @ArchTest
    static final ArchRule transverse_does_not_depend_on_features =
        noClasses().that().resideInAnyPackage("..core..", "..security..", "..shared..")
            .should().dependOnClassesThat().resideInAPackage("..feature..");

    @ArchTest
    static final ArchRule no_cycles_between_features =
        slices().matching("..feature.(*)..").should().beFreeOfCycles();

    @ArchTest
    static final ArchRule no_field_injection =
        noFields().should().beAnnotatedWith(Autowired.class);
}
```

Décision à valider — ArchUnit ou Spring Modulith (D10)

Recommandé : ArchUnit seul au départ (simple, indépendant du framework). Spring Modulith (`ApplicationModules.of(ShopApplication.class).verify()`) ajoute la vérification de modules, les événements persistants et la documentation générée ; à envisager quand le nombre de features dépasse ~8. Modulith traite les sous-packages d'une feature comme internes : l'interface `service/` doit alors être exposée via `@NamedInterface`.

5. Design patterns utilisés

Un pattern se justifie par un problème rencontré, jamais par principe. Si le code reste lisible sans lui, on ne l'introduit pas.

5.1 Strategy

Plusieurs algorithmes interchangeables (calcul de remise, export PDF/CSV). Spring injecte toutes les implémentations ; on les indexe, on ne fait pas de `switch` sur un type.

✓ Strategy avec injection de la liste

```
public interface DiscountPolicy {
    DiscountType type();
    BigDecimal apply(BigDecimal amount);
}

@Service
public class DiscountService {
    private final Map<DiscountType, DiscountPolicy> policies;

    public DiscountService(List<DiscountPolicy> all) {
        this.policies = all.stream().collect(Collectors.toUnmodifiableMap(DiscountPolicy::type,
Function.identity()));
    }

    public BigDecimal apply(DiscountType type, BigDecimal amount) {
        var policy = policies.get(type);
        if (policy == null) throw new IllegalArgumentException("Remise inconnue : " + type);
        return policy.apply(amount);
    }
}
```

5.2 Factory / Builder

- **Factory statique** (`Money.of(...)`, `Order.create(...)`) quand la construction impose des invariants ; le constructeur reste privé ou package-private.
- **Builder** (Lombok `@Builder` ou builder manuel) au-delà de 4 paramètres d'un même type ; pour les DTO, préférer un `record`.

5.3 Mapper et Adapter

Entité ↔ DTO via MapStruct (`componentModel = "spring"`, `unmappedTargetPolicy = ERROR`). Un client d'API externe est isolé derrière une interface (`PaymentGateway`) : l'implémentation HTTP est un adapter remplaçable en test.

5.4 Événements applicatifs

Pour découpler deux features (« une commande est confirmée → envoyer un email »), on publie un événement. Le consommateur ne doit s'exécuter qu'**après** le commit de la transaction émettrice.

✓ **Listener après commit**

```
public record OrderConfirmedEvent(UUID orderId, UUID customerId) {}

// Publication (dans le service de la feature order, transaction en cours)
events.publishEvent(new OrderConfirmedEvent(order.getId(), order.getCustomerId()));

// Consommation (feature notification)
@Component
class OrderConfirmedEventListener {
    @Async
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    void on(OrderConfirmedEvent event) {
        mailer.sendConfirmation(event.orderId());
    }
}
```

✗ **@EventListener + @Async : pas lié au commit**

```
@EventListener
@Async
void on(OrderConfirmedEvent event) { ... }   // s'exécute avant le commit, peut envoyer un email pour une commande annulée ensuite
```

Correction V2.1 — Événement et transaction

La V2 utilisait `@EventListener` + `@Async`. Si la transaction échoue après la publication, l'effet de bord a déjà eu lieu. V2.1 impose `@TransactionalEventListener(AFTER_COMMIT)`.

Pour les effets critiques (paiement, facturation) où la perte d'un événement est inacceptable : pattern **outbox** (table `outbox_event` écrite dans la même transaction, relayée par un job) ou Spring Modulith event publication registry.

5.5 Specification

Une règle de filtrage métier devient un objet nommé, réutilisable et composable. Spring Data JPA 4 fournit `PredicateSpecification<T>` (filtre seul, composable avec `and`/`or`/`not`) et `JpaSpecificationExecutor<T>` ; `Specification<T>` reste utilisable quand l'accès à la `CriteriaQuery` est nécessaire.

✓ **PredicateSpecification composable (Spring Data JPA 4)**

```
public final class OrderSpecs {
    private OrderSpecs() {}

    public static PredicateSpecification<Order> active() {
        return (from, cb) -> cb.equal(from.get("status"), OrderStatus.ACTIVE);
    }
    public static PredicateSpecification<Order> totalAtLeast(BigDecimal min) {
        return (from, cb) -> cb.greaterThanOrEqualTo(from.get("totalAmount"), min);
    }
}

// Composition
var spec = OrderSpecs.active().and(OrderSpecs.totalAtLeast(new BigDecimal("1000")));
Page<Order> page = orders.findAll(spec, pageable);
```

5.6 Filtres et chaîne de traitement

Les filtres servlet et la `SecurityFilterChain` forment un pipeline : l'ordre est critique et doit être explicite. Chaque filtre traite, délègue ou court-circuite.

Ordre	Étape	Implémentation Spring
1	Identifiant de corrélation + MDC	Filtre `OncePerRequestFilter` en tête de chaîne
2	Limitation de débit	Filtre Bucket4j (ou reverse proxy)
3	CORS	`CorsConfigurationSource` (avant l'authentification)

Ordre	Étape	Implémentation Spring
4	Authentification	Filtre JWT (`BearerTokenAuthenticationFilter`)
5	Autorisation	`authorizeHttpRequests` + `@PreAuthorize`
6	Gestion globale des erreurs	`@RestControllerAdvice` + `AuthenticationEntryPoint` / `AccessDeniedHandler` au format `ProblemDetail`

Pour un traitement transverse lié à Spring MVC (audit des appels, mesure de durée), utiliser un `HandlerInterceptor` ; jamais de logique métier dans un filtre.

5.7 Autres patterns à connaître

Pattern	Usage	Quand l'utiliser
Builder	Construction d'objets complexes	Beaucoup de paramètres optionnels
Decorator	Ajout de comportement dynamique	Cache, logging, retry autour d'un service
Observer	Notification de changements	Événements applicatifs (§5.4)
Mediator	Découplage commande/handler	CQRS, pipelines de traitement (à n'introduire qu'avec un besoin réel)
Chain of Responsibility	Chaîne de traitement	Validations en cascade, filtres
Adapter	Conversion d'interface	Intégration de bibliothèques tierces (§5.3)
Proxy	Contrôle d'accès à un objet	Transactions, cache, sécurité (Spring AOP)
Singleton	Instance unique	Géré par le conteneur Spring (scope `singleton` par défaut)

5.8 Patterns à éviter

- Singleton manuel : Spring gère déjà le cycle de vie des beans.
- Repository générique « fourre-tout » : un repository par agrégat/entité racine, avec des méthodes nommées par intention.
- Classes `*Manager`, `*Helper`, `*Utils` : voir §1 (nommage). Les packages `core/` et `shared/` ne contiennent que des types précis.

6. Clean code et Java moderne

6.1 Principes

- **Lisibilité d'abord** : un lecteur comprend une méthode sans ouvrir une autre. Méthode ≤ 30 lignes, classe ≤ 300 lignes (seuils en §2, mesurés par Checkstyle).
- **Un niveau d'abstraction par méthode** ; early return plutôt que `else` imbriqués (profondeur ≤ 3).
- **Pas de nombre magique** : constante nommée ou propriété de configuration.
- **Commentaires** : expliquent le *pourquoi*, jamais le *quoi*. Pas de code commenté (Git s'en souvient).
- **Duplication** : à partir de 3 occurrences d'un bloc de plus de 10 lignes (ou 2 si la logique métier est identique), on factorise. Un seul seuil dans tout le document.

> **Correction V2.1 — Seuil de duplication**
> La V2 donnait « 20 lignes » en §2 et « 10 lignes » en §6. V2.1 retient **10 lignes** partout.

6.2 Fonctionnalités Java 21 à utiliser

Fonctionnalité	Usage VVS	Exemple
`record`	DTO, Value Objects (`Money`, `Address`), événements, propriétés de config	`public record OrderRDto(UUID id, OrderStatus status) {}`
`sealed interface`	Résultats fermés (succès / échec métier)	`sealed interface PaymentResult permits Paid, Declined`
Pattern matching `switch`	Remplace `instanceof` en cascade ; exhaustif sur sealed/enum	`return switch (r) { case Paid p -> ...; case Declined d -> ...; };`
Text blocks	SQL / JSON de test, jamais de concaténation de requête	`""" SELECT ... """`
`var`	Uniquement si le type est évident à droite ; jamais sur un appel de méthode opaque	`var orders = new ArrayList<Order>();`
`Stream`	Transformations courtes ; au-delà de 4 opérations ou d'effets de bord, boucle `for`	—
Collections séquencées	`getFirst()`, `getLast()`, `reversed()` (Java 21)	`list.getLast()`

6.3 Optional, null et JSpecify

- `Optional` : **type de retour uniquement** (jamais paramètre, champ ni élément de collection). Ne jamais appeler `.get()` sans `isPresent()` : utiliser `orElseThrow(() -> new NotFoundException(...))`.
- Une collection retournée n'est jamais `null` : liste vide.
- Spring Boot 4 / Framework 7 adoptent **JSpecify**. Chaque package porte un `package-info.java` avec `@NullMarked` ; seuls les éléments nullables sont annotés `@Nullable` (`org.jspecify.annotations`). Remplace `@NonNull` de Lombok / `javax`.

✓ **package-info.java**

```
@NullMarked
package com.vvs.shop.feature.order.service;
```

6.4 Argent, dates et temps

> **Règle — Argent**
>
> `BigDecimal` (jamais `double`/`float`), échelle fixée (2 pour EUR), `RoundingMode.HALF_UP`, en base `NUMERIC(19,2)`. Pour une devise multiple : type valeur `Money(BigDecimal amount, Currency currency)`. Pour MGA (ariary), préciser l'échelle avec le métier.

> **Règle — Dates**
>
> Instants (création, modification, expiration) : `Instant` en code, `TIMESTAMPTZ` en base, **UTC** partout, sérialisé ISO-8601 (`2026-10-13T07:50:00Z`). Dates sans heure (anniversaire) : `LocalDate`. Jamais `Date`, `Calendar` ni `LocalDateTime` pour un instant.
>
> Ne jamais appeler `Instant.now()` dans le code métier : injecter un `Clock` (testable avec `Clock.fixed`). Fuseau d'affichage appliqué par le frontend.

6.5 Concurrence et threads virtuels

Spring Boot 3.2+ propose `spring.threads.virtual.enabled=true` (l'exécuteur applicatif et le serveur web passent aux threads virtuels). Intérêt : I/O bloquantes nombreuses (appels HTTP, JDBC) sans pool de threads énorme.

> **Décision à valider — Threads virtuels (D11)**
>
> **Recommandé** : activés par défaut sur les nouveaux projets Java 21+, avec ces garde-fous : taille du pool Hikari dimensionnée (les threads virtuels ne multiplient pas les connexions), pas de `ThreadLocal` volumineux. Sur Java 21, éviter `synchronized` autour d'I/O (il épingle le thread porteur ; préférer `ReentrantLock`) ; depuis Java 24 (JEP 491) ce problème disparaît pour `synchronized`, il reste pour les appels natifs. À mesurer avant généralisation sur un projet existant.

Règles pour le code asynchrone :

- **Aucun appel externe sans timeout** : client HTTP (`connectTimeout`, `readTimeout`), JDBC (`connection-timeout`), appels entre services. Un appel sans délai maximal est un défaut P1.
- Pas de `.get()` / `.join()` sans délai sur un `CompletableFuture` dans le thread de requête ; pas de `Thread.sleep` hors tests de bas niveau.
- `@Async` toujours avec un `TaskExecutor` nommé et dimensionné (jamais le défaut implicite) et un `AsyncUncaughtExceptionHandler` : une exception ne doit jamais disparaître silencieusement.
- Le contexte (MDC `correlationId`, `SecurityContext`) est propagé explicitement vers les threads asynchrones (`TaskDecorator`, `DelegatingSecurityContextExecutor`).
- Pas de « fire and forget » sans journalisation d'échec ; pour les travaux longs ou critiques : file / outbox (§5.4), pas un thread lancé à la main.

6.6 Gestion des erreurs

- Exceptions **métier** non vérifiées héritant de `BusinessException` (`NotFoundException`, `ConflictException`, `ValidationException`) ; pas d'exceptions vérifiées dans les signatures de service.
- Ne jamais avaler une exception (`catch` vide) ; ne jamais logger **et** relancer (double log).
- Les messages d'erreur exposés au client ne contiennent ni stack trace, ni SQL, ni chemin de fichier (OWASP A10:2025).
- Un seul `@RestControllerAdvice` par application convertit les exceptions en réponse d'erreur (§7.4). **Les controllers ne contiennent jamais de `try/catch`.**

- Toujours conserver la cause : `throw new InfrastructureException("Échec d'envoi", e)` (jamais `throw new X(e.getMessage())` qui perd la stack trace). Relancer tel quel avec `throw e;`.
- Pas d'exception pour le flux normal ; une erreur **prévisible** du flux métier se modélise par un résultat typé (ci-dessous). Jamais `throw new RuntimeException("...")` : toujours une exception typée de la hiérarchie.

Exception	Statut	Usage
`BusinessException` (racine)	422	Règle métier violée, donnée valide mais incohérente
`NotFoundException`	404	Ressource introuvable (ou non visible par l'utilisateur)
`UnauthorizedException`	401	Non authentifié
`ForbiddenException`	403	Authentifié mais permission insuffisante
`ConflictException`	409	Doublon, contrainte d'unicité ou d'état violée
Autre `Exception`	500	Bug : journalisé en `ERROR`, message générique renvoyé

Pattern Result pour les erreurs prévisibles : un `sealed interface` rend les cas exhaustifs, vérifiés par le compilateur.

✓ Result avec sealed interface

```
public sealed interface RegistrationResult {
    record Registered(UUID userId)      implements RegistrationResult {}
    record EmailAlreadyUsed(String email) implements RegistrationResult {}
    record WeakPassword(List<String> reasons) implements RegistrationResult {}
}

// Consommation : switch exhaustif, aucun oubli possible
return switch (service.register(request)) {
    case Registered r        -> ResponseEntity.created(location(r.userId())).build();
    case EmailAlreadyUsed e  -> throw new ConflictException("Email déjà utilisé");
    case WeakPassword w      -> throw new BusinessException(w.reasons());
};
```

Correction V2.1 — Statut des erreurs métier

422 pour toute donnée invalide ou règle métier violée (une seule famille), **400** uniquement pour une requête illisible (§7.4).

6.7 Logging

- SLF4J : `private static final Logger log` (ou `@Slf4j` si Lombok est retenu, D7) ; jamais `System.out` ni `printStackTrace`.
- Paramètres `{}` plutôt que concaténation ; niveaux : `ERROR` (action requise), `WARN` (anormal mais géré), `INFO` (événement métier), `DEBUG` (diagnostic, désactivé en prod).
- Jamais de mot de passe, token, numéro de carte ni donnée personnelle brute dans les logs. Identifier par `userId`.
- Un `correlationId` (MDC, propagé via header `X-Request-Id`) sur chaque ligne ; format JSON en production.
- Journaliser les **événements métier importants** (création, validation, paiement) en `INFO`, avec identifiants et non données personnelles. Journaliser une erreur avec l'exception complète : `log.error("Échec envoi {}", id, e)`.

Niveau	Usage	Exemple
`TRACE`	Débogage fin (désactivé hors diagnostic)	Entrée/sortie de méthode
`DEBUG`	Débogage général (désactivé en prod)	Valeurs de variables intermédiaires
`INFO`	Événement métier normal	`Commande {} créée`, `Utilisateur {} connecté`
`WARN`	Situation anormale non bloquante	`Échec de connexion pour l'utilisateur {}` (identifiant, jamais l'e-mail)
`ERROR`	Erreur à traiter (récupérable ou non)	`Échec d'envoi d'email à {}`

7. Conception de l'API REST

7.1 URL et verbes

Opération	Verbe + URL	Succès	Corps
Lister	GET `/api/v1/orders?page=1&size=20&sortBy=createdAt&sortDirection=DESC`	200	`PageResponse<OrderRDto>`
Lire	GET `/api/v1/orders/{id}`	200	`OrderRDto`
Créer	POST `/api/v1/orders`	201 + header `Location`	`OrderRDto`
Remplacer	PUT `/api/v1/orders/{id}`	200	`OrderRDto`
Modifier partiellement	PATCH `/api/v1/orders/{id}`	200	`OrderRDto`
Supprimer	DELETE `/api/v1/orders/{id}`	204	aucun
Sous-ressource	GET `/api/v1/orders/{id}/lines`	200	`PageResponse<OrderLineRDto>`
Action métier	POST `/api/v1/orders/{id}/confirm`	200	`OrderRDto`
Export	GET `/api/v1/orders/export`	200 (fichier)	flux binaire

- Ressources au **pluriel**, kebab-case, noms et non verbes. Pas de `/getOrders`.
- Identifiants en **UUID** exposés ; jamais l'identifiant séquentiel interne (énumération, A01).
- Version dans l'URL (`/api/v1`). Une rupture de contrat = `/api/v2` ; les changements additifs ne cassent pas la version.

> **Décision à valider — Versionnement (D12)**
> Spring Framework 7 apporte un support natif du versionnement d'API (`@GetMapping(version = "1")`). **Recommandé** : rester sur le préfixe d'URL `/api/v1` (visible, simple pour Angular et la doc) ; réévaluer si le besoin de versions par header apparaît.

7.2 DTO d'entrée et de sortie

- `XxxWDto` (write) pour la création/modification, `XxxRDto` (read) pour la lecture, `XxxDto` quand lecture et écriture ont la même forme. Les mêmes suffixes sont utilisés côté Angular (modèle `OrderRDto`), voir §10.
- Tous les DTO sont des `record`. Validation Jakarta sur le DTO (`@NotBlank`, `@Size`, `@Email`, `@Positive`) + `@Valid` dans le controller.
- **Jamais** d'entité JPA dans une signature de controller (fuite de schéma, lazy loading, mass assignment).
- Les champs sensibles (`passwordHash`, `internalNotes`) n'existent pas dans les RDto.

✓ **DTO d'écriture validé**

```
public record CreateOrderWDto(
    @NotNull UUID customerId,
    @NotEmpty @Size(max = 100) List<@Valid OrderLineWDto> lines,
    @Size(max = 500) String note) {}

public record OrderLineWDto(@NotNull UUID productId, @Positive int quantity) {}
```

7.3 Pagination

`Page<T>` de Spring Data ne doit pas être sérialisée telle quelle (structure instable, avertissement Spring Data). On renvoie un record stable. Le **contrat VVS** est commun à tous les services, pour qu'un même client Angular puisse les consommer : paramètres `page` (**base 1**), `size`, `search`, `sortBy`, `sortDirection` ; réponse `items`, `totalCount`, `page`, `pageSize`, `totalPages`.

✓ PageResponse et requête de pagination

```
public record PageResponse<T>(List<T> items, int page, int pageSize, long totalCount, int totalPages)
{
    public static <T> PageResponse<T> from(Page<T> p) {
        return new PageResponse<>(p.getContent(), p.getNumber() + 1, p.getSize(),   // base 1 pour le
client
                                  p.getTotalElements(), p.getTotalPages());
    }
}

public record PageQuery(
    @Min(1) Integer page, @Min(1) @Max(100) Integer size,
    @Size(max = 100) String search, String sortBy, SortDirection sortDirection) {

    public PageQuery {                                    // valeurs par défaut si absentes de l'URL
        page = page == null ? 1 : page;
        size = size == null ? 20 : size;
    }

    public Pageable toPageable(Set<String> allowedSortFields) {
        var field = sortBy != null && allowedSortFields.contains(sortBy) ? sortBy : "createdAt"; //
liste blanche
        var dir = sortDirection == SortDirection.ASC ? Sort.Direction.ASC : Sort.Direction.DESC;
        return PageRequest.of(page - 1, size, Sort.by(dir, field));
    }
}
```

Réponse paginée

```
{ "items": [ ... ], "totalCount": 1234, "page": 1, "pageSize": 20, "totalPages": 62 }
```

- Taille par défaut 20, **maximum 100**. **Tout** endpoint GET qui retourne une liste est paginé : un endpoint sans pagination qui renvoie 100 000 lignes est un incident de production (P0).
- Champs de tri en liste blanche (jamais le nom de colonne brut issu de l'utilisateur).
- Grands volumes ou flux temps réel : pagination par curseur (`after=<id>`) ; à documenter dans l'ADR de la feature.

Décision à valider — Contrat de pagination (D20)

Recommandé : contrat VVS (page base 1, `items` / `totalCount`) : lisible, indépendant de Spring Data, identique pour tous les services. Alternative officielle : `PagedModel` de Spring Data (`@EnableSpringDataWebSupport(pageSerializationMode = VIA_DTO)`, page base 0, `content` + `page.totalElements`) — zéro code, mais le front dépend du format Spring.

7.4 Format des erreurs

Correction V2.1 — Un seul format d'erreur

La V2 mélangeait une enveloppe `ApiResponse<T>` pour les succès et un format d'erreur différent, alors que le frontend Angular type directement `OrderRDto`. V2.1 : **les succès renvoient la ressource brute (pas d'enveloppe)** ; seules les erreurs ont un corps standardisé, au format **RFC 9457 `application/problem+json`** (`ProblemDetail` de Spring).

Décision à valider — Format d'erreur (D13)

Recommandé : `ProblemDetail` (standard, natif Spring) avec extensions `code` (machine) et `errors` (champs). Alternative : conserver `ApiResponse<T>` partout — plus uniforme mais pénalise la lisibilité des appels et du typage Angular.

✓ **Réponse d'erreur**

```
{
  "type": "https://api.vvs.mg/problems/validation",
  "title": "Validation échouée",
  "status": 422,
  "detail": "1 champ invalide",
  "instance": "/api/v1/orders",
  "code": "VALIDATION_FAILED",
  "correlationId": "7f1c0b9e",
  "errors": [{ "field": "lines[0].quantity", "message": "doit être supérieur à 0" }]
}
```

Cas	Statut	code
JSON illisible, paramètre de type invalide	400	`MALFORMED_REQUEST`
Non authentifié / jeton invalide	401	`UNAUTHENTICATED`
Authentifié mais interdit	403	`FORBIDDEN`
Ressource introuvable (ou non visible par l'utilisateur)	404	`NOT_FOUND`
Conflit d'état (doublon, version optimiste, transition interdite)	409	`CONFLICT`
Validation Jakarta ou règle métier sur les données	422	`VALIDATION_FAILED`
Trop de requêtes	429 + `Retry-After`	`RATE_LIMITED`
Erreur non prévue	500 (message générique)	`INTERNAL_ERROR`

> **Correction V2.1 — Statuts 400 / 409 / 422**
>
> La V2 renvoyait 400 ou 409 pour une validation selon les exemples. V2.1 fixe : **422** pour les données invalides, **409** pour les conflits d'état, **400** réservé aux requêtes syntaxiquement incorrectes.

✓ **Gestionnaire global**

```
@RestControllerAdvice
class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(NotFoundException.class)
    ProblemDetail notFound(NotFoundException ex) {
        return problem(HttpStatus.NOT_FOUND, "NOT_FOUND", ex.getMessage());
    }

    @ExceptionHandler(Exception.class)
    ProblemDetail unexpected(Exception ex) {
        log.error("Erreur non prévue", ex);                // détail uniquement dans les logs
        return problem(HttpStatus.INTERNAL_SERVER_ERROR, "INTERNAL_ERROR", "Erreur interne");
    }
    // handleMethodArgumentNotValid(...) surchargé → 422 + errors[]
}
```

7.5 Routes : bonnes et mauvaises pratiques

✕ Incorrect	Pourquoi	✓ Correct
GET `/api/getOrders`	Verbe dans l'URL	GET `/api/v1/orders`
POST `/api/createOrder`	Verbe dans l'URL	POST `/api/v1/orders`
GET `/api/order`	Singulier	GET `/api/v1/orders/{id}`

✕ Incorrect	Pourquoi	✓ Correct
POST `/api/orders/delete/123`	Utiliser le verbe DELETE	DELETE `/api/v1/orders/123`
GET `/api/Orders`	PascalCase	kebab-case minuscule
GET `/api/order_lines`	snake_case	GET `/api/v1/order-lines`

7.6 Controllers minces

Un controller est une coquille : il reçoit la requête HTTP, la délègue à l'**interface** de service et renvoie la réponse. **Maximum 5 lignes par action, zéro logique métier, aucun `try/catch`, aucun accès repository, aucune entité.**

✓ Controller mince

```
@RestController
@RequestMapping("/api/v1/orders")
@Validated
@Tag(name = "Orders")
public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) { this.orderService = orderService; }

    @PostMapping
    @HasPermission("orders.create")          // méta-annotation de §9.3
    public ResponseEntity<OrderRDto> create(@Valid @RequestBody CreateOrderWDto dto) {
        var created = orderService.create(dto);
        return ResponseEntity.created(URI.create("/api/v1/orders/" + created.id())).body(created);
    }
}
```

✕ Controller gras (interdit)

```
@PostMapping
public ResponseEntity<?> create(@RequestBody CreateOrderWDto dto) {
    if (dto.lines() == null || dto.lines().isEmpty())           // ✕ validation dans le controller
        return ResponseEntity.badRequest().body("Lignes manquantes");
    var order = new Order();                                     // ✕ mapping manuel
    orderRepository.save(order);                                 // ✕ repository direct, pas de
transaction
    return ResponseEntity.ok(order);                             // ✕ entité exposée
}
```

Annotations attendues sur chaque controller : `@RestController`, `@RequestMapping("/api/v1/...")`, `@Validated`, une règle d'autorisation (`@PreAuthorize` ou configuration d'URL), et la documentation OpenAPI des statuts (annotation `@ApiResponse` de springdoc/OpenAPI, sans rapport avec l'enveloppe D13 : 200/201, 400, 401, 403, 404, 422).

7.7 Idempotence, ETag et documentation

- PUT, DELETE et GET sont idempotents. Pour un POST critique (paiement), accepter un header `Idempotency-Key`.
- Concurrence optimiste : champ `@Version` sur les entités modifiables ; conflit → 409.
- OpenAPI généré par springdoc (`/v3/api-docs`), exposé uniquement hors production ou derrière authentification ; le client Angular est généré depuis ce contrat quand c'est possible.
- Sérialisation JSON avec **Jackson 3** (`tools.jackson`, `JsonMapper`) ; les annotations (`@JsonProperty`) restent dans `com.fasterxml.jackson.annotation`. Champs inconnus ignorés en lecture, dates ISO-8601.

8. Persistance : JPA et PostgreSQL

8.1 Entités

- Dans `<feature>/domain/`, sans suffixe `Entity` (`Order`, pas `OrderEntity`). Jamais `@Data` sur une entité : `@Getter` + méthodes métier ciblées ; `equals/hashCode` sur l'identifiant, jamais sur toute la classe.
- Un constructeur `protected` sans argument (exigé par JPA, non `private`) + factory statique portant les invariants.
- `@Enumerated(EnumType.STRING)` toujours (jamais `ORDINAL`).
- Associations : `fetch = LAZY` par défaut, y compris `@ManyToOne`. Jamais d'`EAGER`.
- Pas d'association JPA entre features (§2, D8) : on stocke l'`UUID` de l'autre entité.
- `BaseEntity` (`@MappedSuperclass`) : `createdAt`, `updatedAt`, `createdBy`, `updatedBy` renseignés par l'audit JPA (§8.7), jamais à la main. L'identifiant est déclaré dans chaque entité (UUIDv7, §8.3).
- `open-in-view` désactivé : `spring.jpa.open-in-view=false`. Le mapping vers DTO se fait dans la transaction du service.

✓ Entité FBL (JPA dans le domaine de la feature)

```
@Entity
@Table(name = "orders")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class Order extends BaseEntity {

    @Id
    @UuidGenerator(style = UuidGenerator.Style.VERSION_7)   // Hibernate 7, voir 8.3
    private UUID id;

    @Column(name = "customer_id", nullable = false)
    private UUID customerId;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private OrderStatus status;

    @Column(name = "total_amount", nullable = false, precision = 19, scale = 2)
    private BigDecimal totalAmount;

    @Version
    private long version;

    public static Order create(UUID customerId, BigDecimal total) {
        if (total.signum() < 0) throw new BusinessException("Montant négatif");
        var o = new Order();
        o.customerId = Objects.requireNonNull(customerId);
        o.totalAmount = total;
        o.status = OrderStatus.DRAFT;
        return o;
    }
}
```

8.2 Nommage en base

Élément	Règle	Exemple
Table	snake_case, pluriel ; `orders` (et non `order`, mot réservé SQL)	`orders`, `order_lines`
Colonne	snake_case, singulier	`customer_id`, `created_at`
Clé primaire	`id`	—

Élément	Règle	Exemple
Clé étrangère (colonne)	`<table_singulier>_id`	`customer_id`
Contraintes	`pk_`, `fk_`, `uq_`, `ck_` + table + colonnes	`uq_users_email`
Index	`ix_<table>_<colonnes>`	`ix_orders_customer_id_created_at`
Booléens	préfixe `is_`/`has_`	`is_active`

Stratégie Hibernate : `PhysicalNamingStrategy` snake_case (par défaut Spring) ; les noms sont écrits explicitement dans `@Table`/`@Column` pour qu'un renommage de champ Java ne modifie pas silencieusement le schéma.

8.3 Identifiants

> **Décision à valider — Type d'identifiant (D14)**
> **Recommandé** : UUID générés par l'application, colonne `uuid`. Préférer **UUIDv7** (ordonné dans le temps, bien meilleur pour les index B-tree que le v4 aléatoire) : `@UuidGenerator(style = UuidGenerator.Style.VERSION_7)` avec Hibernate 7 côté Java ; la fonction SQL `uuidv7()` est native à partir de PostgreSQL 18 (pour les insertions hors application). Alternative : `BIGINT GENERATED ALWAYS AS IDENTITY` en interne + UUID public (`public_id`) si la performance d'index prime.

> **Règle — Multi-société et multi-boutique**
> Si le produit peut un jour servir plusieurs sociétés ou boutiques (cas d'un ERP), toute table métier porte dès sa création une colonne `company_id` (et `shop_id` si pertinent), indexée, filtrée de façon centralisée. L'ajouter plus tard touche toutes les tables et toutes les requêtes.

8.4 Migrations de base de données

- Un seul outil de migration par dépôt (D24) : Flyway par défaut, Liquibase si le projet l'utilise déjà. Le schéma est **uniquement** géré par cet outil : `spring.jpa.hibernate.ddl-auto=validate` (jamais `update` ni `create` hors tests jetables).
- Dépendances (Spring Boot 4) : `spring-boot-starter-flyway` **et** `flyway-database-postgresql` (obligatoire depuis Flyway 10), ou `spring-boot-starter-liquibase`. La bibliothèque seule ne suffit plus à déclencher l'auto-configuration.
- Flyway : fichiers `src/main/resources/db/migration/V<AAAAMMJJHHmm>__description.sql` (horodatage plutôt qu'un compteur : pas de collision entre branches). Si une branche fusionne une migration plus ancienne que la dernière appliquée, Flyway la refuse à la validation : la renommer avec un horodatage plus récent avant la fusion ; ne pas activer `outOfOrder` en production. Liquibase : convention de nommage du projet (`<séquence>__<feature>__<description>.sql`, `includeAll`).
- Une migration appliquée n'est **jamais modifiée** ; on en crée une nouvelle. Une migration = un changement cohérent.
- Changements compatibles avec la version précédente de l'appli (expand / contract) : ajout de colonne nullable → backfill → contrainte NOT NULL dans une migration ultérieure. Pas de `DROP COLUMN` dans la même release que l'arrêt de son usage.
- Données de référence versionnées en migration ; données de démo seulement dans le profil `dev` (`db/dev`).

✓ Migration

```
-- V202610131030__create_orders.sql
CREATE TABLE orders (
    id            uuid          PRIMARY KEY,
    customer_id   uuid          NOT NULL,
    status        varchar(20)   NOT NULL,
    total_amount  numeric(19,2) NOT NULL,
    created_at    timestamptz   NOT NULL,
    updated_at    timestamptz   NOT NULL,
    created_by    uuid,
    updated_by    uuid,
    version       bigint        NOT NULL DEFAULT 0,
    CONSTRAINT ck_orders_total_positive CHECK (total_amount >= 0)
);
CREATE INDEX ix_orders_customer_id_created_at ON orders (customer_id, created_at DESC);
```

8.5 Requêtes et performance

- **N+1** : détecter en test (compteur de requêtes) ; corriger par `@EntityGraph`, `JOIN FETCH` ou projection DTO. Jamais en activant l'`EAGER`.
- Listes : projections (record / interface) plutôt que d'entités complètes. Recherche dynamique : Specifications ou Querydsl ; jamais de concaténation de chaînes.
- `@Query` avec paramètres nommés (`:email`), jamais d'interpolation (injection SQL, A05).
- Chaque clé étrangère et chaque colonne de filtre/tri fréquent est indexée ; vérifier avec `EXPLAIN (ANALYZE, BUFFERS)` sur un jeu de données réaliste avant une release.
- Opérations en lot : `saveAll` + `hibernate.jdbc.batch_size`, ou `JdbcClient` pour les imports lourds.
- Pool Hikari : `maximum-pool-size` explicite (défaut 10) ; `connection-timeout` court ; surveiller `hikaricp_connections_pending`.

Spécifications : filtres de recherche composés avec `PredicateSpecification<T>` (§5.5), jamais de requêtes assemblées par concaténation. **Pas d'accès direct** à `EntityManager` ou `JdbcTemplate` depuis un service métier : on passe par un repository (interface, éventuellement avec implémentation personnalisée) (violation P0).

8.6 Transactions

- Transactions déclarées sur la couche **service** : `@Transactional(readOnly = true)` sur la classe, `@Transactional` sur les méthodes d'écriture.
- Jamais d'appel réseau lent (HTTP, email) à l'intérieur d'une transaction : utiliser l'événement `AFTER_COMMIT` (§5.4).
- Un appel `this.methode()` interne ne passe pas par le proxy : ne pas compter sur `@Transactional` pour un appel interne.
- **Unit of Work** : la transaction du service **est** l'unité de travail (le `EntityManager` regroupe les modifications et les écrit au commit). Interdit : appeler `saveAndFlush` après chaque modification dans une méthode transactionnelle ; on modifie les entités gérées et on laisse le commit écrire.

8.7 Audit, horodatage et suppression logique

Audit, horodatage et suppression logique sont du comportement transversal : ils ne se codent pas dans les services métier.

Besoin	Mécanisme Spring / Hibernate
`createdAt/By`, `updatedAt/By`	`@EnableJpaAuditing` + `@EntityListeners(AuditingEntityListener.class)` + `AuditorAware<UUID>`
Historique Insert/Update/Delete	Hibernate Envers (`@Audited`) ou listener écrivant dans une table `audit_log`

Besoin	Mécanisme Spring / Hibernate
Suppression logique	`@SQLDelete(sql = "UPDATE ... SET is_deleted = true WHERE id = ?")` + `@SQLRestriction("is_deleted = false")`

- L'écriture d'audit utilise une **transaction séparée** (`REQUIRES_NEW`) ou une table dédiée, pour éviter qu'une écriture d'audit ne déclenche elle-même l'audit (boucle) et qu'un échec d'audit n'annule pas la transaction métier sans décision explicite.
- La suppression logique est une décision par entité (données légales ou traçables) ; elle impose un index partiel `WHERE is_deleted = false` sur les colonnes uniques.

8.8 Tests de persistance

Testcontainers avec **la même image PostgreSQL que la production** (jamais H2 : dialecte, JSONB, contraintes et index diffèrent). Voir §11.

9. Sécurité

9.1 Correspondance avec l'OWASP Top 10:2025

OWASP 2025	Mesures VVS
A01 Broken Access Control (inclut SSRF)	Autorisation par défaut « refuser » ; contrôle de propriété sur chaque ressource (`order.customerId == currentUser`), pas seulement le rôle ; UUID exposés ; URL sortantes en liste blanche.
A02 Security Misconfiguration	Profils séparés, secrets hors dépôt, `actuator` restreint, CORS explicite, en-têtes de sécurité, Swagger fermé en prod.
A03 Software Supply Chain Failures	Dépendances verrouillées, Dependabot, `dependency-check`/SCA en CI, images Docker épinglées par version (digest en prod), SBOM (CycloneDX).
A04 Cryptographic Failures	TLS partout, Argon2id (bcrypt en héritage), secrets JWT forts, aucune donnée sensible en clair en base ni en logs.
A05 Injection	Requêtes paramétrées, validation d'entrée, encodage de sortie ; Angular échappe par défaut (jamais `bypassSecurityTrust*` sans revue).
A06 Insecure Design	Modélisation des menaces légère par feature sensible (ADR), limites de débit, règles métier côté serveur.
A07 Authentication Failures	Verrouillage, limitation de débit sur login, message d'erreur unique, rotation des refresh tokens, MFA pour les comptes administrateurs.
A08 Software or Data Integrity Failures	Pas de désérialisation d'objets non fiables ; CI à droits minimaux ; artefacts signés ou vérifiés.
A09 Security Logging and Alerting Failures	Événements de sécurité journalisés (login échoué, refus d'accès, changement de rôle) avec alerte, pas seulement des logs.
A10 Mishandling of Exceptional Conditions	Gestionnaire d'erreurs global, échec fermé (fail closed), pas de fuite de détails, pas de `catch` vide.

> **Correction V2.1 — Référentiel mis à jour**
> La V2 citait la numérotation OWASP 2021 (A06 « Vulnerable Components », A08, etc.). V2.1 s'aligne sur 2025 : « Components » devient **Software Supply Chain Failures (A03)**, SSRF est intégré à A01, et **Mishandling of Exceptional Conditions (A10)** est nouveau.

9.2 Authentification et jetons

Élément	Règle
Hachage des mots de passe	Jamais MD5, SHA-256 simple ni texte clair. Argon2id recommandé (OWASP : `Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8()`, dépendance BouncyCastle requise ; OWASP : au minimum 19 Mio, 2 itérations, parallélisme 1) ; bcrypt accepté en héritage (coût ≥ 10, limite de 72 octets du mot de passe). Toujours via `DelegatingPasswordEncoder` (préfixe `{id}`) pour migrer sans casse ; minimum 8 caractères avec MFA, 15 sans (NIST SP 800-63B, repris par OWASP), longueur maximale ≥ 64, pas de règle de composition forcée, refus des mots de passe courants.
Access token JWT	Durée 15 minutes, algorithme explicite (HS256 avec secret ≥ 256 bits, ou RS256/EdDSA si plusieurs services), claims `sub`, `iat`, `exp`, `jti`, `roles`, `permissions` ; tolérance d'horloge (clock skew) ≤ 30 s à la validation.
Refresh token	Opaque (aléatoire 256 bits), stocké haché en base, durée 7 jours, rotation à chaque usage ; la réutilisation d'un ancien token révoque toute la famille.

Élément	Règle
Stockage côté navigateur	Refresh token en cookie `HttpOnly; Secure; SameSite=Strict` ; access token en mémoire (jamais `localStorage`).
Verrouillage de compte	Après N tentatives échouées (5 par défaut, configurable), compte verrouillé pour une durée croissante ; compteur remis à zéro au succès ; événement de sécurité journalisé (A09).
Déconnexion	Révocation du refresh token côté serveur.
Secrets	Fournis par variables d'environnement ou secret manager ; jamais dans `application.yml` versionné.

Correction V2.1 — Tolérance d'horloge

La V2 imposait à la fois « aucune tolérance » et « 60 s » selon les passages. V2.1 : **30 secondes maximum**, valeur unique dans `JwtProperties`.

Spring Security tolère par défaut 60 s de dérive d'horloge à la validation d'un JWT. Décision D21 : conserver une tolérance de ≤ 30 s (recommandé) ou 0 s (strict, suppose des serveurs synchronisés par NTP).

Décision à valider — Cookie ou en-tête pour le refresh (D15)

Recommandé : refresh en cookie HttpOnly + protection CSRF sur l'endpoint `/auth/refresh` (SameSite=Strict + vérification `Origin`). Alternative pour une API consommée par mobile : refresh en corps de réponse, stocké dans le trousseau sécurisé de l'appareil.

9.3 Autorisation

- Une `SecurityFilterChain` explicite ; règle finale `anyRequest().authenticated()` (jamais `permitAll` global).
- Rôles stockés sans préfixe (`ADMIN`, `MANAGER`, `USER`) ; `@EnableMethodSecurity` activé. Le code teste des **permissions** (ci-dessous) ; `hasRole('ADMIN')` est réservé aux opérations d'administration.
- **Conversion des claims** : un `JwtAuthenticationConverter` configuré explicitement — claim `permissions` sans préfixe (`setAuthorityPrefix("")`), claim `roles` préfixé `ROLE_`. Sans cela, Spring Security lit le claim `scope` avec le préfixe `SCOPE_` et les contrôles ne correspondent jamais.
- **Permissions fines** : claims `permissions` dans le JWT (`orders.create`, `orders.read`…) et méta-annotation `@HasPermission("orders.create")`, définie avec la **méta-annotation à gabarit** de Spring Security : `@PreAuthorize("hasAuthority('{value}')")` et un bean `AnnotationTemplateExpressionDefaults`. Les rôles regroupent des permissions ; le code teste des permissions.
- Seuls `login`, `register`, `forgot-password`, `refresh` et la santé (`/actuator/health`) sont publics ; tout le reste est authentifié par défaut.
- Contrôle de propriété dans le service : charger la ressource **filtrée par utilisateur** (`findByIdAndCustomerId`) ; renvoyer 404 plutôt que 403 pour ne pas révéler l'existence.
- Mass assignment : les DTO d'entrée n'ont pas de champ `role`, `id`, `ownerId`.

9.4 CORS, CSRF, en-têtes

- **CORS** : liste blanche d'origines par profil (`https://app.vvs.mg`), jamais `*` avec credentials ; méthodes et en-têtes explicites.
- **CSRF** : désactivé uniquement si l'API est sans état et que l'authentification ne repose pas sur un cookie ; sinon activé (ou SameSite=Strict + vérification d'origine) — cohérent avec D15.
- **Sanitisation** : toute entrée utilisateur qui sera rendue en HTML est assainie côté serveur (OWASP Java HTML Sanitizer) ou stockée telle quelle et échappée à l'affichage ; jamais de HTML brut non assaini.

- En-têtes : `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `Content-Security-Policy` stricte côté frontend, `Referrer-Policy`, `frame-ancestors none`.

9.5 Limitation de débit et abus

- Login, reset de mot de passe, inscription, envoi d'email : limitation par IP **et** par identifiant (Bucket4j ou reverse proxy), réponse 429 + `Retry-After`.
- Taille maximale des requêtes et des fichiers (`spring.servlet.multipart.max-file-size`) ; type MIME vérifié côté serveur ; fichiers stockés hors du répertoire servi, noms régénérés.

9.6 Données sensibles

- Chiffrer en base les champs réellement sensibles (jeton tiers, pièce d'identité) ; le reste est protégé par chiffrement du disque et des sauvegardes.
- Minimisation et durée de conservation définies par feature ; export et suppression des données personnelles possibles sur demande.
- Les environnements de dev/test n'utilisent jamais de données de production non anonymisées.

10. Frontend Angular / TypeScript

10.1 Principes

- **Composants standalone** uniquement (pas de `NgModule`), `ChangeDetectionStrategy.OnPush` explicite (devenu le défaut des nouvelles applications en v22, mais on l'écrit pour la lisibilité et les projets v21).
- **Zoneless** : `provideZonelessChangeDetection()` (défaut en v21). Aucune dépendance à `zone.js` ; l'état qui déclenche l'affichage est un **signal**.
- **Signals** pour l'état local et dérivé : `signal()`, `computed()`, `effect()` (réservé aux effets de bord, jamais pour propager de l'état). Entrées/sorties : `input()`, `output()`, `model()`.
- **RxJS** pour les flux asynchrones d'événements (recherche avec debounce, websocket) ; conversion `toSignal()` en bordure de composant. Pas de `subscribe()` manuel dans un composant sans `takeUntilDestroyed()`.
- Nouveau flux de contrôle `@if`, `@for` (avec `track` obligatoire), `@switch` ; plus de `*ngIf`/`*ngFor`.
- `inject()` plutôt que l'injection par constructeur ; services `providedIn: 'root'` pour le transverse, fournis dans la route pour une feature.
- `strict: true`, `strictTemplates: true`, jamais de `any` (utiliser `unknown` + garde de type).

10.2 Appels HTTP et état serveur

Les services d'accès aux données vivent dans `features/<feature>/data-access/`. Les types sont ceux du backend (`OrderRDto`, `CreateOrderWDto`, `PageResponse<T>`), sans enveloppe (voir §7.4).

✓ **Service d'accès aux données**

```
@Injectable({ providedIn: 'root' })
export class OrderService {                 // fichier order-service.ts (§1.2)
  private readonly http = inject(HttpClient);
  private readonly base = '/api/v1/orders';

  list(page: number, size = 20): Observable<PageResponse<OrderRDto>> {
    return this.http.get<PageResponse<OrderRDto>>(this.base, { params: { page, size } });
  }
  create(body: CreateOrderWDto): Observable<OrderRDto> {
    return this.http.post<OrderRDto>(this.base, body);
  }
}
```

> **Décision à valider — resource() / httpResource (D16)**
> Angular 22 stabilise `resource()` et `httpResource()` (lecture réactive d'une ressource HTTP exposée en signaux : `value()`, `isLoading()`, `error()`). **Recommandé** : pour les lectures simples, `httpResource` ; pour les écritures et les flux complexes, `HttpClient` + RxJS. Sur Angular 21, ces API sont expérimentales : ne pas les utiliser en production.

10.3 Formulaires

> **Décision à valider — Reactive Forms ou Signal Forms (D17)**
> **Recommandé** : **Reactive Forms typés** (`FormBuilder.nonNullable`) comme standard actuel. Signal Forms est stabilisé en Angular 22 mais récent ; autorisé sur les nouvelles fonctionnalités après une preuve de concept dans l'équipe, sans mélanger les deux approches dans un même formulaire.

- La validation client est un confort ; la **source de vérité est le serveur** : les erreurs 422 (`errors[].field`) sont réaffectées aux champs du formulaire.
- Chaque champ a un `<label>` associé ; les erreurs sont annoncées (`aria-live`, `aria-describedby`). Bouton d'envoi désactivé pendant la requête (anti double soumission).

10.4 Routage, guards, interceptors

- Routes **lazy** par feature (`loadChildren` / `loadComponent`), configuration dans `<feature>.routes.ts`.
- Guards, interceptors et resolvers **fonctionnels** (`CanActivateFn`, `HttpInterceptorFn`) ; fichiers `auth-guard.ts` / `auth-interceptor.ts` selon le schéma de la CLI v20+ (séparateur tiret pour ces trois types).
- Interceptors : `authInterceptor` (ajoute le jeton, tente un refresh sur 401 une seule fois), `errorInterceptor` (convertit `ProblemDetail` en erreur typée, notification globale), `correlationInterceptor`.
- Pas de logique d'autorisation uniquement côté client : le guard améliore l'expérience, le serveur décide.

✓ **Interceptor fonctionnel**

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthStore).accessToken();
  const isApiCall = req.url.startsWith('/api/');      // jamais de jeton vers un domaine tiers
  const authReq = token && isApiCall ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
  : req;
  return next(authReq);
};
```

10.5 Gestion d'état

- Niveau 1 : signaux dans le composant. Niveau 2 : **store de feature** (classe injectable avec signaux privés en écriture et `asReadonly()` en lecture). Niveau 3 : NgRx SignalStore seulement si plusieurs features partagent un état complexe — décision d'équipe documentée en ADR.
- Jamais d'état global mutable dans un module/fichier ; jamais de `BehaviorSubject` public exposé en écriture.

10.6 Accessibilité, i18n, performance

- **Accessibilité (WCAG 2.2 AA)** : éléments HTML sémantiques (`button`, `nav`, `main`), focus visible, navigation au clavier, contraste ≥ 4,5:1, textes alternatifs. Contrôle automatisé (axe) dans les tests e2e.
- **i18n** : tous les textes visibles passent par la traduction (`@angular/localize` ou ngx-translate, un seul outil par projet) ; langues du produit : fr, mg (et en au besoin) ; formats de date/nombre via les locales.
- **Performance** : `NgOptimizedImage`, `@defer` pour les blocs lourds, budgets de bundle dans `angular.json` (échec du build au-delà), `track` sur chaque `@for`.
- Sécurité : pas de `innerHTML` avec contenu utilisateur ; jamais `bypassSecurityTrust*` sans revue ; pas de secret dans le bundle (tout ce qui est dans le front est public).

11. Tests

11.1 Pyramide et outils

Niveau	Backend	Frontend	Part visée
Unitaire	JUnit 5, Mockito, AssertJ	Vitest + Angular Testing Library / TestBed	~70 % (pyramide 70/20/10)
Intégration	`@SpringBootTest` / `@DataJpaTest` / `@WebMvcTest` + Testcontainers PostgreSQL	Tests de composants avec `HttpTestingController`	~25 %
Bout en bout	Parcours critiques via API (RestAssured / MockMvc)	Playwright (parcours critiques + axe)	~5 %

> **Correction V2.1 — Karma retiré du standard**
> La V2 prescrivait Karma + Jasmine. Depuis Angular 21, **Vitest est le runner par défaut** et Karma n'est plus le choix standard. V2.1 : Vitest pour les nouveaux projets ; migration progressive des projets Karma existants (schematic de la CLI).

11.2 Backend

- Nom des tests : `methode_scenario_resultatAttendu` (§1.1), p. ex. `createOrder_emptyLines_throwsBusinessException`, `login_lockedAccount_throwsBadCredentials`, `calculateTotal_withDiscount_appliesCorrectly`. Structure **Arrange / Act / Assert** (ou Given / When / Then) séparée par des commentaires ; **un test = un scénario = une assertion logique**.
- Le système testé s'appelle `sut` (System Under Test). Les tests sont indépendants et exécutables dans n'importe quel ordre.
- Tests unitaires : aucune dépendance externe (base, réseau, fichier) et **< 100 ms** chacun ; couvrir cas positifs, négatifs et cas limites. Cas multiples : `@ParameterizedTest` + `@CsvSource` / `@MethodSource`.
- Mocks : `@MockitoBean` / `@MockitoSpyBean` (Spring Framework 6.2+) ; `@MockBean` / `@SpyBean` sont supprimés de Spring Boot 4.
- Dépendances de test Spring Boot 4 : starters par technologie (`spring-boot-starter-data-jpa-test`, `spring-boot-starter-webmvc-test`…) ; `@SpringBootTest` ne fournit plus MockMvc seul : ajouter `@AutoConfigureMockMvc`.
- Les tests d'intégration utilisent `@ServiceConnection` avec un conteneur PostgreSQL **de même version qu'en production** ; un conteneur partagé par suite (réutilisation) pour la vitesse.
- Temps : `Clock.fixed(...)` injecté. Aléatoire : graine fixe. Aucun `Thread.sleep` : `Awaitility`.
- Sécurité testée : accès sans jeton (401), mauvais rôle (403), ressource d'un autre utilisateur (404), mass assignment ignoré.
- Tests ArchUnit (§4.3) exécutés avec les tests unitaires.

✓ **Test d'intégration avec Testcontainers**

```
@Testcontainers
@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
class OrderRepositoryIT {

    @Container @ServiceConnection
    // Testcontainers 2.x : org.testcontainers.postgresql.PostgreSQLContainer (non générique)
    static PostgreSQLContainer pg = new PostgreSQLContainer("postgres:18-alpine");

    @Autowired OrderRepository orders;

    @Test
    void findByCustomerId_returnsOnlyThatCustomersOrders() {
        // given
        var mine = orders.save(Order.create(CUSTOMER_A, BigDecimal.TEN));
        orders.save(Order.create(CUSTOMER_B, BigDecimal.TEN));
        // when
        var page = orders.findByCustomerId(CUSTOMER_A, PageRequest.of(0, 20));
        // then
        assertThat(page.getContent()).extracting(Order::getId).containsExactly(mine.getId());
    }
}
```

11.3 Frontend

- On teste le **comportement visible** (texte, rôles ARIA, événements), pas les détails internes ; sélection par rôle/label plutôt que par classe CSS.
- Composants zoneless : `await fixture.whenStable()` après une action ; pas de `fakeAsync` sauf nécessité.
- HTTP simulé avec `provideHttpClientTesting()` ; chaque test vérifie `httpMock.verify()`.

11.4 Couverture et qualité des tests

> **Décision à valider — Seuil de couverture (D18)**
>
> **Recommandé** : couverture de lignes **≥ 80 %** sur les services/règles métier (JaCoCo, Vitest coverage), sans seuil global imposé sur les getters/DTO ; une PR ne doit pas faire baisser la couverture du code modifié. Optionnel : mutation testing (PIT) sur les modules critiques.

Un test instable (flaky) est corrigé ou supprimé sous 48 h ; il ne reste jamais ignoré (`@Disabled`) sans ticket.

12. Outillage qualité et intégration continue

Cette section est nouvelle dans la V2.1 : une convention qui n'est pas appliquée automatiquement n'est pas respectée.

Domaine	Outil	Rôle	Bloquant en CI
Format Java	Spotless (google-java-format, indentation 4 espaces — voir note)	Formatage unique	Oui
Règles Java	Checkstyle + PMD (ou SonarQube)	Tailles, nommage, complexité cyclomatique ≤ 10, imports inutiles et `*`, `System.out`, `UnsupportedOperationException("TODO")`	Oui (erreurs)
Compilation	`-Xlint:all -Werror` / TypeScript strict	0 avertissement ; imports inutilisés = erreur (Checkstyle / Spotless)	Oui
Architecture	ArchUnit	Dépendances entre couches/features	Oui
Format front	Prettier	Formatage unique	Oui
Règles front	ESLint (angular-eslint + typescript-eslint, config stricte)	Qualité, accessibilité des templates	Oui
Dépendances	Dependabot / Renovate + audit (`npm audit`, OWASP dependency-check)	Mises à jour et vulnérabilités	Oui (critiques)
Secrets	gitleaks (pre-commit + CI)	Aucun secret commité	Oui
Conteneurs	Trivy sur les images	Vulnérabilités OS/lib	Oui (critiques)
SBOM	CycloneDX	Inventaire des composants (A03)	Non

> **Décision à valider — Style de formatage Java (D19)**
> google-java-format impose l'indentation de 2 espaces. Pour garder 4 espaces (usage courant, cohérent avec le style de ce document) : Spotless avec Eclipse formatter ou palantir-java-format. **Recommandé** : palantir-java-format via Spotless (4 espaces, lignes ≤ 120 caractères).

12.1 Pipeline minimal (par Pull Request)

Étape	Contenu	Échec ⇒
1. Lint & format	Spotless check, Checkstyle, ESLint, Prettier check, gitleaks	PR bloquée
2. Build & tests unitaires	Maven/Gradle `verify`, Vitest	PR bloquée
3. Intégration	Testcontainers (PostgreSQL), ArchUnit	PR bloquée
4. Sécurité	Dependency-check / npm audit, Trivy (image)	PR bloquée si critique
5. Build image Docker	Image taguée par SHA court, non poussée sur PR	PR bloquée
6. Déploiement (branche principale)	Push image → déploiement environnement de recette	Alerte équipe

- La branche principale est **protégée** : PR obligatoire, CI verte, 1 approbation minimale, force-push interdit.
- Les versions d'actions CI sont épinglées (SHA ou version exacte) ; permissions du token CI au minimum (`contents: read`).

- Les caches (Maven/Gradle/npm) sont activés pour garder la CI sous 10 minutes.

13. Workflow Git et revue de code

13.1 Branches

Type	Format	Exemple
Fonctionnalité	`feature/<ticket>-<description-kebab>`	`feature/VTE-01-vente-multicanal`
Correction	`fix/<ticket>-<description>`	`fix/STK-04-alerte-seuil`
Maintenance	`chore/<description>`	`chore/upgrade-spring-boot`
Branche principale	`main` (toujours déployable)	—

13.2 Commits et Pull Requests

- **Conventional Commits** : `feat(order): ajoute la pagination`, `fix(auth): corrige la validation d'expiration`, `refactor`, `test`, `docs`, `chore`, `perf`. Corps : le *pourquoi*. Rupture de compatibilité : `!` après le type (`feat(api)!:`) ou pied de page `BREAKING CHANGE:`.
- Une PR = un objectif ; taille cible **≤ 400 lignes modifiées** (hors fichiers générés et migrations) ; au-delà, découper.
- Titre de PR et de ticket : `ID · SPx · Titre court` (convention projet). Le corps de la PR référence l'issue GitHub avec `Refs #<numéro>` (et non `Closes #ID`, qui fermerait le ticket avant validation).
- Fusion par **squash merge** vers `main` ; en cours de développement, on met à jour sa branche par **merge** de `main` (pas de rebase d'une branche partagée).
- Pas de commit direct sur `main`, pas de `--force` sur une branche partagée, pas de fichier généré ou de secret commité.

13.3 Revue de code

- Délai de première réponse : **1 jour ouvré**. L'auteur relit sa propre PR avant de demander la revue.
- Le relecteur vérifie d'abord la correction et la sécurité (P0), puis la conception (P1), puis le style (P2) — voir la grille en §15. Le style est du ressort des outils, pas des humains.
- Un commentaire bloquant est préfixé `P0:` / `P1:` ; les suggestions facultatives `nit:`. Le désaccord se règle à deux, puis avec le référent technique.

14. Configuration, Docker et observabilité

Section nouvelle dans la V2.1.

14.1 Configuration

- Profils : `dev`, `test`, `prod`. `application.yml` (commun) + `application-<profil>.yml`. Aucun secret versionné : variables d'environnement ou secret manager.
- Propriétés typées : `@ConfigurationProperties` + `record` + `@Validated` (échec au démarrage si une valeur obligatoire manque) — pas de `@Value` dispersé.
- Principe « twelve-factor » : la même image tourne en dev, recette et production ; seule la configuration change.

✓ **Propriétés typées**

```
@ConfigurationProperties(prefix = "vvs.jwt")
@Validated
public record JwtProperties(
    @NotBlank String secret,
    @NotNull Duration accessTtl,
    @NotNull Duration refreshTtl,
    @DurationMin(seconds = 0) @DurationMax(seconds = 30) Duration clockSkew) {}
```

14.2 Docker

- Dockerfile **multi-étapes**, image d'exécution minimale (JRE 21 ou 25 « jlink »/distroless), exécution en **utilisateur non root**, sonde de santé sur `/actuator/health/liveness` (définie dans Compose ou l'orchestrateur : l'image JRE ne contient pas forcément `curl`), version d'image épinglée (pas de `latest`).
- Java en conteneur : `-XX:MaxRAMPercentage=75`, pas de `-Xmx` en dur. Optionnel : image par Buildpacks (`spring-boot:build-image`).
- `docker-compose.yml` de développement : PostgreSQL (version figée, volume nommé), backend, frontend, outils (pgAdmin/Mailpit) ; `docker compose up` suffit à démarrer le projet. Spring Boot Docker Compose support autorisé en `dev`.
- `.dockerignore` obligatoire ; aucun secret dans l'image ni dans les couches.
- Poste de développement Windows : Docker Engine dans WSL2, projet cloné **dans le système de fichiers WSL** (performances).

✓ **Dockerfile multi-étapes**

```
FROM eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY mvnw pom.xml ./
COPY .mvn .mvn
# couche de dépendances, mise en cache tant que pom.xml ne change pas
RUN ./mvnw -q dependency:go-offline
COPY src src
RUN ./mvnw -q -DskipTests package \
 && cp target/*.jar application.jar \
 && java -Djarmode=tools -jar application.jar extract --layers --destination /layers

FROM eclipse-temurin:21-jre
RUN useradd --system --uid 10001 vvs
USER 10001
WORKDIR /app
COPY --from=build /layers/dependencies/ ./
COPY --from=build /layers/spring-boot-loader/ ./
COPY --from=build /layers/snapshot-dependencies/ ./
COPY --from=build /layers/application/ ./
EXPOSE 8080
# santé : sonde /actuator/health/liveness définie dans Compose ou l'orchestrateur
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "application.jar"]
```

14.3 Observabilité

- **Actuator** : exposer uniquement `health` (liveness/readiness) et `info` publiquement ; `metrics`/`prometheus` sur un port de management interne ou authentifié.
- **Logs** JSON en production avec `correlationId` ; **métriques** Micrometer (latence, taux d'erreur, pool Hikari, JVM) ; **traces** via `spring-boot-starter-opentelemetry` (Spring Boot 4).
- Alertes minimales : disponibilité, taux d'erreurs 5xx, latence p95, saturation du pool de connexions, échecs de login anormaux (A09).
- Sauvegardes PostgreSQL automatiques **et** restauration testée au moins une fois par trimestre.

15. Grille de revue et checklist de Pull Request

15.1 Grille par priorité

Grille de revue en trois niveaux de gravité (P0 / P1 / P2), adaptée à Java / Spring / Angular.

Priorité	Signification	Critères
P0 — bloquant, fusion impossible	Bug, faille ou perte de données possible	Violation de l'architecture : code dans la mauvaise couche, dépendance interdite (§4), controller épais (§7.6), `EntityManager` / `JdbcTemplate` injecté dans un service métier Faille de sécurité : contrôle d'accès manquant (A01), injection (SQL, XSS), énumération d'emails, mot de passe en clair, secret ou donnée sensible dans le code ou les logs Entité JPA exposée par l'API · requête construite par concaténation · migration modifiée après application · `EAGER` / N+1 évident sur une liste `UnsupportedOperationException("TODO")` ou `NotImplemented` dans une PR · `System.out` / `printStackTrace` au lieu de SLF4J · code qui ne compile pas ou compile avec warnings Endpoint GET qui renvoie une liste sans pagination · `catch` vide qui avale l'exception · données codées en dur au lieu de service / configuration Duplication de code > 10 lignes · test absent sur une règle métier critique
P1 — à corriger avant la fusion (sous 24 h)	Conception et maintenabilité	DTO d'entrée sans validation Jakarta · pas de logger dans un service · valeurs en dur au lieu de `@ConfigurationProperties` · controller sans règle d'autorisation Appel externe sans timeout · `@Async` sans executor ni gestion d'erreur · `Instant.now()` direct · `Optional` mal utilisé Exception non typée (`new RuntimeException("...")` au lieu de `BusinessException`) · statut HTTP ou format d'erreur incohérent Violation de SOLID ou du nommage (§1, §3, §7) · méthode > 30 lignes sans justification · composant Angular sans `OnPush` ou `track` manquant Pas de test unitaire sur un service métier · requête de lecture sans projection ni `readOnly`
P2 — suggestion	Style et confort	Commentaire inutile ou sur-verbeux · code mort ou commenté · import inutile (si non détecté par l'outil) Nommage améliorable · classe > 300 lignes · Javadoc manquante sur une méthode publique critique

15.2 Checklist de l'auteur (modèle de description de PR)

Modèle .github/pull_request_template.md

```
## Quoi / Pourquoi
Refs #ID

## Vérifications
- [ ] Compile sans warning ; CI verte (format, lint, tests, ArchUnit, sécurité)
- [ ] Architecture : code dans la bonne couche, controllers minces (≤ 5 lignes), constructeurs sur
interfaces
- [ ] Chaque DTO d'entrée validé · logs SLF4J sur les événements importants · timeouts sur les appels
externes
- [ ] Configuration typée (@ConfigurationProperties), aucune valeur en dur
- [ ] Tests ajoutés/mis à jour (cas nominal, erreur, droits)
- [ ] Aucun secret, aucune entité exposée dans l'API
- [ ] Migration (Flyway ou Liquibase) ajoutée, jamais modifiée, et compatible avec la version
précédente
- [ ] Pagination bornée, statuts HTTP et format d'erreur conformes (§7)
- [ ] Textes traduits, accessibilité vérifiée (si front)
- [ ] Documentation / ADR mise à jour si décision structurante
- [ ] Taille de la PR raisonnable (≤ 400 lignes), une seule intention

## Comment tester
...
```

Annexe A — Journal des corrections V2 → V2.1

A.1 Faits techniques obsolètes ou inexacts

#	V2	V2.1	Pourquoi
1	Spring Boot 4.0.x	Cible 4.1.x	Le support open source de la 4.0 s'arrête le 31/12/2026 ; la 4.1 est disponible.
2	Angular « 20/21 » sans précision	Angular 21 ou 22 (D3)	v22 sortie en juin 2026 ; Signal Forms et `httpResource` y sont stables.
3	Tests front Karma + Jasmine	Vitest	Vitest est le runner par défaut depuis Angular 21.
4	OWASP Top 10:2021	OWASP Top 10:2025	Nouvelle numérotation : supply chain (A03), exceptions (A10), SSRF fusionné dans A01.
5	`@MockBean`	`@MockitoBean`	Déprécié en Boot 3.4, supprimé en Boot 4.
6	Packages Jackson `com.fasterxml`	Jackson 3 : `tools.jackson`	Annotations inchangées (`com.fasterxml.jackson.annotation`).
7	Flyway seul	+ `flyway-database-postgresql`	Module obligatoire depuis Flyway 10.
8	`@Valid` seulement	+ JSpecify `@NullMarked`	Spring Boot 4 / Framework 7 standardisent la null-safety.

A.2 Contradictions internes corrigées

#	Contradiction dans la V2	Décision V2.1
9	Suffixes DTO différents entre §Java et §Angular	Mêmes noms partout : `RDto` / `WDto` / `Dto` (§7.2, §10.2)
10	Valeurs d'enum en `PascalCase` ou `snake_case` selon les exemples	`UPPER_SNAKE_CASE` en Java ; stocké en `STRING` (§1, §8.1)
11	Règle « pas de point dans un nom de fichier » vs `*.spec.ts`	Alignement suffixe fichier ↔ suffixe de symbole (§1) ; `.spec.ts` et `.routes.ts` admis
12	Interdiction des `Utils`/`Manager` mais packages `util/`, `manager/`	Packages supprimés ; classes nommées par responsabilité (§1, §5.8)
13	Seuil de duplication 20 lignes (§2) vs 10 lignes (§6)	10 lignes partout (§6.1)
14	Enveloppe `ApiResponse<T>` vs typage direct Angular	Ressource brute + `ProblemDetail` pour les erreurs (§7.4, D13)
15	Statut 400 / 409 / 422 incohérents pour la validation	422 validation · 409 conflit · 400 requête mal formée (§7.4)
16	Tolérance d'horloge JWT : 0 s vs 60 s	≤ 30 s, unique (§9.2)
17	`Page<T>` sérialisé tel quel	`PageResponse<T>` (§7.3)

A.3 Exemples de code corrigés

#	Problème dans la V2	Correction
18	Listener `@EventListener` + `@Async` (exécuté avant le commit)	`@TransactionalEventListener(AFTER_COMMIT)` + outbox pour les effets critiques (§5.4)
19	Login : fuite d'existence du compte (temps/message), pas de verrouillage	Hash factice, message unique, verrouillage (§4.2)

#	Problème dans la V2	Correction
20	`security/` dépendait de la feature `user`	Port `AuthenticatedUserLoader` côté transverse (§4.2)
21	Exemple ISP : entités et `List` non paginée, paramètres à une lettre	DTO, `Page`, noms explicites (§3.4)
22	Type générique `TEntity`, imports manquants dans les exemples	Génériques `T`/`ID` conventionnels ; exemples compilables
23	`@Transactional` sur méthodes sans `readOnly`	Classe en `readOnly = true`, écriture surchargée (§2, §8.6)

A.4 Bonnes pratiques renforcées en V2.1

#	Règle renforcée	Intégration VVS (Java)
R1	Controllers minces : ≤ 5 lignes, zéro logique, zéro `try/catch`	§7.6, grille P0
R2	Pattern `Result<T>` pour les erreurs prévisibles	§6.6 (`sealed interface` + `switch` exhaustif)
R3	Hiérarchie d'exceptions ↔ statuts HTTP, cause toujours conservée	§6.6 (tableau, conservation de la cause)
R4	Règles async : timeouts, pas de blocage, propagation du contexte	§6.5
R5	Specification, `@ConfigurationProperties`, filtres, autres patterns	§5.5, §5.6, §5.7, §14.1
R6	Unit of Work, pas d'`EntityManager` dans les services, audit / soft delete / horodatage	§8.5, §8.6, §8.7
R7	Contrat de pagination et routes (sous-ressources, actions, export)	§7.1, §7.3, §7.5 (D20)
R8	Verrouillage après N échecs, `@HasPermission`, sanitisation HTML	§9.2, §9.3, §9.4
R9	Tests : `sut`, AAA, < 100 ms, un assert logique, 70/20/10, 80 %	§11
R10	Grille P0 / P1 / P2 et checklist PR, 0 warning, imports ordonnés	§2.2, §12, §15

A.5 Sections ajoutées

- §0 Socle technique et versions · §6.2–6.5 Java 21, JSpecify, argent/dates, threads virtuels · §8 PostgreSQL (nommage, UUID, Flyway, performance)
- §9.1 correspondance OWASP 2025 · §9.2 politique de jetons · §9.5 limitation de débit · §10.3–10.6 formulaires, état, accessibilité, i18n
- §12 Outillage qualité et CI · §13 Workflow Git · §14 Configuration, Docker, observabilité · Annexes A à E (choix d'architecture, Clean Architecture)

Annexe B — Décisions à valider et points à vérifier

Chaque décision a une option **recommandée** (appliquée dans le document) ; si l'équipe en choisit une autre, la section concernée est à ajuster. À traiter en une réunion d'équipe d'environ une heure.

B.1 Décisions de l'équipe

ID	Question	Recommandation V2.1	Alternative	§
D1	Java 21 ou 25 LTS ?	21 minimum, 25 pour les nouveaux projets	25 partout	0
D2	Version majeure de PostgreSQL ?	18 pour les nouveaux projets (`uuidv7()` natif) ; 16 ou 17 conservés si déjà en place (Valimbavaka : 16)	Monter tous les projets en 18	0, 8
D3	Angular 21 ou 22 ?	22 pour les nouveaux projets (TypeScript 6 requis) ; 21 maintenu (LTS jusqu'en 06/2027)	21 partout	0, 10
D4	Maven ou Gradle ?	Maven (déjà connu de l'équipe), wrapper versionné	Gradle	0
D5	Suffixe des classes Angular (`OrderList` vs `OrderListComponent`)	Sans suffixe (guide de style Angular récent)	Avec suffixe	1
D6	Préfixe des sélecteurs	`vvs-` (ou préfixe projet)	Préfixe par projet	1
D7	Lombok autorisé ?	Oui, périmètre limité (voir §2)	Interdit, records + IDE	2
D8	Associations JPA entre features	Interdites (UUID), dérogation par ADR	Autorisées	2, 8
D9	Emplacement des services d'accès aux données Angular	Dans la feature (`data-access/`)	`core/` global	2, 10
D10	ArchUnit ou Spring Modulith	ArchUnit	Modulith	4
D11	Threads virtuels	Activés sur les nouveaux projets, après mesure	Désactivés	6
D12	Versionnement d'API	Préfixe `/api/v1`	Versionnement natif par header	7
D13	Format d'erreur et enveloppe	RFC 9457 `ProblemDetail`, succès sans enveloppe	`ApiResponse<T>` partout (`success`, `message`, `data`, `timestamp`)	7
D14	Identifiants	UUID (v7 si possible)	BIGINT interne + `public_id`	8
D15	Stockage du refresh token	Cookie HttpOnly + SameSite=Strict	Corps de réponse (mobile)	9
D16	`httpResource` / `resource()`	Lectures simples uniquement, Angular 22	`HttpClient` seul	10
D17	Reactive Forms ou Signal Forms	Reactive Forms typés (standard)	Signal Forms (POC)	10
D18	Seuil de couverture	≥ 80 % sur logique métier	Seuil global / aucun	11
D19	Formateur Java	palantir-java-format (4 espaces)	google-java-format (2 espaces)	12
D20	Contrat de pagination	Contrat VVS : page base 1, `items` / `totalCount`	Natif Spring (base 0, `content`)	7

ID	Question	Recommandation V2.1	Alternative	§
D21	Tolérance d'horloge JWT	≤ 30 s (Spring : 60 s par défaut)	0 s (strict, suppose des horloges NTP)	9
D22	Annotations Spring dans la couche application (Clean Architecture)	`@Service` / `@Transactional` autorisés (pragmatique)	Application sans Spring, beans déclarés dans `shop-boot`	E
D23	Architecture du projet	FBL par défaut ; Clean Architecture si métier riche, par projet ou par module (Annexe D)	Clean Architecture partout	2, 4, D, E
D24	Outil de migration de base	Un seul par dépôt : Flyway (défaut de la convention) ou Liquibase si déjà en place (cas Valimbavaka)	Changer d'outil	8

B.2 Points à vérifier avant publication

Ces éléments dépendent de versions ou de réalités que le document ne peut pas garantir : à confirmer sur les sources officielles ou sur vos projets réels.

- **Vérifié le 15/10/2026** (sources officielles, Annexe C) : Spring Boot 4.1 (support OSS jusqu'au 31/07/2027, 4.0 jusqu'au 31/12/2026), Angular 22 (3 juin 2026, actif jusqu'en 06/2027, LTS 06/2028), OWASP Top 10:2025, Spring Boot 4 (starters de test, `@MockitoBean`, starters Flyway/Liquibase, MockMvc), Spring Data JPA 4 (`PredicateSpecification`), Spring Security 7 (méta-annotations à gabarit, Argon2), Testcontainers 2 (`org.testcontainers.postgresql`), Dockerfile en couches. **Non vérifié** : compatibilité des bibliothèques tierces ; les extraits dépendant de Spring n'ont pas été compilés.
- **Versions exactes** : dernière 4.1.x de Spring Boot, patch Angular 21/22, version PostgreSQL disponible chez votre hébergeur. Les numéros du §0 sont des cibles, pas des garanties.
- **Compatibilité des bibliothèques** avec Spring Boot 4 / Jackson 3 : MapStruct, Lombok, springdoc, Bucket4j, Testcontainers, jjwt/nimbus. Vérifier le tableau de compatibilité de chacune avant la migration d'un projet existant.
- **Projets existants (Valimbavaka, etc.)** : écart entre le code actuel et cette convention ; établir un plan de migration (ne pas tout réécrire d'un coup — appliquer sur le code nouveau et lors des modifications).
- **Exemples de code** : compiler et exécuter chaque snippet dans un projet modèle (« starter VVS ») avant diffusion ; c'est le meilleur garde-fou contre les erreurs résiduelles.
- **Seuils chiffrés** (30 lignes, 300 lignes, 400 lignes de PR, complexité 10, 80 % de couverture) : cohérents avec la pratique de l'équipe ? Les régler une fois puis les automatiser.
- **Langues et formats** (fr/mg/en, devise MGA/EUR et échelle décimale, fuseaux) : à confirmer avec le métier de chaque produit.
- **Hébergement et conformité** : exigences de résidence des données, politique de conservation, obligations légales applicables — à valider hors équipe technique.
- **Outils CI** : la section §12 est agnostique (GitHub Actions, GitLab CI…) ; choisir la plateforme et écrire le pipeline modèle.
- **Architecture** : FBL par défaut ; Clean Architecture (Annexe E) seulement si l'ADR-0001 du projet ou du module la retient (Annexe D).

B.3 Plan d'adoption proposé

Étape	Action	Responsable
1	Réunion de décision (Annexe B.1) puis mise à jour du document	Coordination technique

Étape	Action	Responsable
2	Créer le dépôt « starter VVS » (backend + frontend + CI + docker-compose) conforme à la convention	Référents back et front
3	Brancher les outils (Spotless, Checkstyle, ArchUnit, ESLint, Prettier, gitleaks) en CI	Référent qualité
4	Appliquer aux nouveaux développements ; migrer l'existant par opportunité	Toute l'équipe
5	Revue de la convention à chaque version majeure de Spring Boot / Angular (au moins 2 fois par an)	Coordination technique

Annexe C — Références

- Spring Boot — politique de support : https://spring.io/projects/spring-boot#support ; dates de fin de support : https://endoflife.date/spring-boot
- Guide de migration Spring Boot 4.0 (starters, tests, Jackson 3, Flyway/Liquibase) : https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide
- Spring Boot — images de conteneur (Dockerfile en couches) : https://docs.spring.io/spring-boot/reference/packaging/container-images/dockerfiles.html
- Spring Security — stockage des mots de passe : https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html · sécurité des méthodes (méta-annotations à gabarit) : https://docs.spring.io/spring-security/reference/servlet/authorization/method-security.html
- Spring Data JPA — Specifications : https://docs.spring.io/spring-data/jpa/reference/jpa/specifications.html
- Angular — calendrier des versions : https://angular.dev/reference/releases · guide de style : https://angular.dev/style-guide
- OWASP Top 10:2025 : https://top10.owasp.org/2025/ · Cheat Sheet Series (mots de passe, JWT, REST) : https://cheatsheetseries.owasp.org
- RFC 9457 — Problem Details for HTTP APIs : https://www.rfc-editor.org/rfc/rfc9457
- Flyway : https://documentation.red-gate.com/flyway · Liquibase : https://docs.liquibase.com
- Testcontainers : https://testcontainers.com · PostgreSQL (versions supportées) : https://www.postgresql.org/support/versioning/
- ArchUnit : https://www.archunit.org · Spring Modulith : https://docs.spring.io/spring-modulith/reference/
- Conventional Commits : https://www.conventionalcommits.org · ADR : https://adr.github.io
- JSpecify : https://jspecify.dev · WCAG 2.2 : https://www.w3.org/TR/WCAG22/

Attention

Les liens ci-dessus sont des points d'entrée : l'état des versions évolue vite. Toujours confirmer sur la source officielle avant de figer une version dans un projet.

Annexe D — Choisir l'architecture et faire évoluer le projet

Règle VVS : FBL par défaut. La Clean Architecture (Annexe E) s'ajoute seulement quand l'ADR du projet — ou d'un module — la justifie. Le choix se fait avant le premier commit et se consigne dans l'ADR-0001.

D.1 Comparaison

Critère	FBL (défaut)	Clean Architecture (Annexe E)
Idée	Découpage par feature métier, couches techniques à l'intérieur	Domaine au centre, dépendances vers l'intérieur
Structure	Un module, packages par feature	Modules Maven : domain, application, infrastructure, api, boot
Entités	Entité JPA = entité métier	Entité de domaine pure + entité JPA + mapper
Coût de départ	Faible	Plus élevé (ports, commandes/résultats, mappers)
Testabilité du métier	Bonne (Mockito sur les repositories)	Excellente : domaine testable sans Spring ni base
Remplacer JPA / broker / IdP	Intrusif	Localisé dans un adaptateur
Courbe d'apprentissage	Douce	Plus raide
Risque principal	Services fourre-tout, entités anémiques	Sur-ingénierie, code de copie entre couches
Garde-fous	ArchUnit (puis Spring Modulith)	Modules Maven + enforcer + ArchUnit

D.2 Questionnaire (10 questions)

Chaque réponse « Clean » vaut 1 point. Répondre pour le projet réel, pas pour l'idéal.

#	Question	FBL	« Clean »
1	Règles métier riches et changeantes ?	Surtout du CRUD et des validations	Beaucoup de règles qui évoluent
2	Durée de vie prévue ?	< 3 ans	> 5 ans
3	Équipe ?	1 à 5, stable	> 5 ou forte rotation
4	Canaux d'entrée ?	REST seulement	REST + batch + messages
5	Intégrations externes volatiles ?	Peu ou stables	Nombreuses ou volatiles
6	Besoin de tester le métier sans base ni Spring ?	Tests d'intégration suffisants	Oui, forte couverture unitaire
7	Niveau en conception (ports, inversion) ?	Débutants / intermédiaires	À l'aise, ou mentor
8	Délai avant la première version ?	Court	Confortable
9	Risque de changer de persistance ou de broker ?	Faible	Réel
10	Le projet va-t-il grossir en modules / équipes ?	Non	Oui

Score « Clean »	Décision
0 à 3	FBL

Score « Clean »	Décision
4 à 6	FBL strict (ArchUnit dès le jour 1, pas d'association JPA entre features), réexamen à chaque jalon
7 à 10	Clean Architecture pour le projet ou le module concerné

Règle

Les questions 1, 5 et 6 pèsent le plus. Si Q1 = « Clean » et Q7 = « Débutants » : FBL, et formation avant de passer à la Clean Architecture.

D.3 Trajectoire d'un projet qui grandit (jusqu'à un ERP)

Un ERP complet ne s'écrit pas avec une seule architecture. La trajectoire VVS est un **monolithe modulaire** : on part en FBL, on regroupe les features en modules quand le projet grossit, et on adopte la Clean Architecture seulement pour les modules à domaine riche.

Étape	Déclencheur	Action
1. FBL strict	Démarrage	ArchUnit dès le premier sprint ; références par identifiant entre features ; pas de cycle ; `company_id` / `shop_id` dans le schéma dès les premières migrations
2. Monolithe modulaire	Environ 8 features ou plus, ou plusieurs équipes	Regrouper les features en modules (`sales`, `inventory`, `accounting`…) ; Spring Modulith pour vérifier les frontières et publier des événements
3. Clean Architecture ciblée	Module à règles riches et changeantes (comptabilité, paie, commissions complexes)	Migrer ce module feature par feature (Annexe E), noter dans un nouvel ADR
4. Extraction éventuelle	Besoin de déployer ou de faire évoluer un module seul	Un module déjà isolé devient un service séparé

Attention

À prévoir tôt car coûteux à rattraper : multi-société / multi-boutique / multi-devise, reporting transversal (module de lecture séparé avec vues SQL), intégrations externes derrière une interface, paramétrage (taxes, workflows, droits fins).

Hors architecture : comptabilité en partie double, paie et règles fiscales sont des projets à part entière, avec des experts métier.

D.4 ADR-0001 à remplir avant de démarrer

Modèle ADR

```
# ADR-0001 — Architecture du projet <nom>

- Statut : proposé | accepté | remplacé par ADR-00NN
- Date : AAAA-MM-JJ
- Décideurs : <noms>

## Contexte
<métier, taille d'équipe, durée de vie, intégrations, contraintes>

## Score du questionnaire (Annexe D.2)
Score « Clean » : N / 10  (Q1=…, Q2=…, …)

## Décision
Architecture retenue : FBL | Clean Architecture (périmètre : projet | module <nom>)

## Raisons
1. <critère qui a tranché>
2. <critère qui a tranché>

## Conséquences acceptées
- <coût ou limite acceptés>

## Condition de réexamen
<ex. : > 8 features, ou 2 intégrations externes critiques, ou module comptabilité>
```

D.5 Migrer de FBL vers Clean Architecture

- Jamais tout d'un coup : feature par feature, en commençant par celle dont le métier est le plus riche.
- Étape 1 : extraire les règles métier dans une classe de domaine sans annotation JPA.
- Étape 2 : créer le port de repository dans le domaine, l'adaptateur JPA, l'entité JPA séparée et son mapper.
- Étape 3 : remplacer le service par des cas d'usage (commande / résultat) ; déplacer le code dans les modules Maven.
- Étape 4 : activer les règles ArchUnit de pureté du domaine pour cette feature ; nouvel ADR.
- Pendant la migration, les deux styles coexistent dans le dépôt : seul cas toléré (ADR obligatoire).

D.6 Pièges

- Choisir la Clean Architecture « pour faire propre » sur un CRUD : on paie le mapping sans bénéfice.
- Choisir FBL puis laisser les controllers appeler les repositories et les features s'importer : on perd tous les avantages.
- Un package ou module « shared » fourre-tout : il ne contient que du technique générique.

Annexe E — Clean Architecture (modules à domaine riche)

Cette annexe décrit la variante **Clean Architecture**. Elle n'est **pas** l'architecture par défaut de VVS : elle s'applique à un projet, ou à un module d'un projet, dont l'ADR l'a retenue (Annexe D) — typiquement la comptabilité, la paie ou un moteur de règles riche. Tout le reste de la convention (nommage, API, sécurité, tests, Git, Docker) s'applique sans changement ; seuls diffèrent la structure du backend, l'exemple de controller et le modèle d'entité.

Principe : séparer ce qui est **stable** (règles métier) de ce qui est **volatil** (framework, base de données, web). Les détails techniques dépendent du métier, jamais l'inverse. Le coût est un surplus de classes et de mapping ; le gain est un domaine testable sans Spring ni base, et des technologies remplaçables.

E.1 Structure et règles

Le code est organisé en **couches concentriques** : le domaine au centre, les cas d'usage autour, les adaptateurs (web, base de données, sécurité, e-mail) à l'extérieur. **Règle de dépendance : le code source ne pointe que vers l'intérieur.** Le domaine ignore Spring, JPA et Jackson. Chaque couche est un **module Maven** distinct : le compilateur interdit alors une dépendance illégale.

Arborescence (modules Maven)

```
vvs-shop (pom parent)
├── shop-domain            // Java pur : ni Spring, ni JPA, ni Jackson
│   └── domain.order       // Order, OrderStatus, OrderRepository (port)
├── shop-application       // cas d'usage ; dépend de domain
│   └── application.order  // CreateOrderUseCase, CreateOrderService,
│                          // CreateOrderCommand, OrderResult
├── shop-infrastructure    // adaptateurs sortants ; dépend de application
│   ├── persistence.order  // OrderJpaEntity, OrderJpaRepository,
│   │                      // OrderRepositoryAdapter, OrderPersistenceMapper
│   └── security           // JwtTokenIssuer, SecurityConfig
├── shop-api               // adaptateur entrant REST ; dépend de application
│   ├── order              // OrderController, OrderRDto, CreateOrderWDto
│   └── error              // GlobalExceptionHandler (ProblemDetail)
└── shop-boot              // @SpringBootApplication, assemblage

Racine des packages : com.vvs.shop.<couche>.<feature>
```

> **Règle**
>
> `shop-api` et `shop-infrastructure` ne se voient pas : l'un appelle des cas d'usage, l'autre implémente des ports. Seul `shop-boot` les connaît tous.
>
> Le domaine contient les entités **métier** (pas d'annotation JPA), les objets valeur et les interfaces de repository (ports sortants).
>
> Les DTO REST (`RDto` / `WDto`) restent dans `shop-api`. Les cas d'usage reçoivent des **commandes** (`CreateOrderCommand`) et renvoient des **résultats** (`OrderResult`) : jamais un DTO HTTP, jamais une entité JPA.

Règles Clean Architecture

- Un cas d'usage = une classe (ou une interface + implémentation) qui orchestre : charger, appeler le domaine, persister, publier un événement. La logique métier vit dans les entités et services du domaine.
- Les annotations Spring de l'application sont limitées à `@Service` et `@Transactional` (décision D22) ; le domaine n'en contient aucune.
- Le mapping est explicite à chaque frontière : DTO ↔ commande/résultat (`shop-api`), entité de domaine ↔ entité JPA (`shop-infrastructure`). MapStruct est autorisé dans les deux adaptateurs.
- **Relations entre features** : référence par identifiant ; une feature appelle une autre via le port entrant de celle-ci, ou via un événement de domaine.

E.2 Couches et sens des dépendances

Couche (module)	Contenu	Dépend de	Ne doit pas connaître
Domain (`shop-domain`)	Entités, objets valeur, règles métier, événements de domaine, ports sortants (`OrderRepository`)	JDK uniquement	Spring, JPA, Jackson, HTTP, SQL
Application (`shop-application`)	Cas d'usage, commandes / résultats, ports entrants et sortants non métier (`PasswordHasher`, `TokenIssuer`)	Domain	JPA, HTTP, Jackson, DTO REST
Infrastructure (`shop-infrastructure`)	Adaptateurs sortants : JPA, sécurité, mail, horloge	Application, Domain	Controllers, DTO REST
API (`shop-api`)	Controllers, DTO, gestionnaire d'erreurs, mapping HTTP	Application	JPA, entités JPA, repositories
Boot (`shop-boot`)	Assemblage, configuration, démarrage	Tous	—

- Flux d'une requête : `Controller → CreateOrderUseCase → Order (domaine) → OrderRepository (port) ⇐ OrderRepositoryAdapter (JPA)`. L'implémentation du port est injectée par Spring depuis `shop-boot`.
- Un port est une **interface appartenant à la couche qui l'utilise** ; l'adaptateur qui l'implémente est dans une couche extérieure (inversion de dépendance).
- Les exceptions métier sont définies dans le domaine ; leur traduction en code HTTP (§7.4 de la convention) n'existe que dans `shop-api`.

E.3 Exemple : authentification via des ports

Le cas d'usage d'authentification dépend de ports (`UserCredentialsPort`, `PasswordHasher`, `TokenIssuer`) ; Spring Security, JWT et JPA restent dans `shop-infrastructure`.

✓ Cas d'usage d'authentification (couche application)

```
// shop-application — ports
public interface UserCredentialsPort {
    Optional<UserCredentials> findByEmail(String email);
    void recordFailure(UUID userId);
    void recordSuccess(UUID userId);
}
public interface PasswordHasher { String hash(String raw); boolean matches(String raw, String hash);
}
public interface TokenIssuer   { TokenPair issue(UserCredentials user); }

public record UserCredentials(UUID id, String passwordHash, Set<String> permissions,
                              boolean enabled, Instant lockedUntil) {}

// shop-application — cas d'usage
@Service
public class LoginService implements LoginUseCase {

    private final UserCredentialsPort users;
    private final PasswordHasher hasher;
    private final TokenIssuer tokens;
    private final Clock clock;
    private final String dummyHash;   // hash factice du même coût, calculé une fois

    public LoginService(UserCredentialsPort users, PasswordHasher hasher, TokenIssuer tokens, Clock
clock) {
        this.users = users; this.hasher = hasher; this.tokens = tokens; this.clock = clock;
        this.dummyHash = hasher.hash("not-a-real-password");
    }

    // Pas de @Transactional : l'échec doit rester enregistré alors qu'une exception est levée.
    @Override
    public TokenPair execute(LoginCommand command) {
        var user = users.findByEmail(command.email().strip().toLowerCase(Locale.ROOT));
        var hash = user.map(UserCredentials::passwordHash).orElse(dummyHash);
        boolean matches = hasher.matches(command.password(), hash);   // toujours exécuté : temps
comparable

        if (user.isEmpty() || !matches || !user.get().enabled() || isLocked(user.get())) {
            user.ifPresent(u -> users.recordFailure(u.id()));
            throw new InvalidCredentialsException();                  // message unique côté API
        }
        users.recordSuccess(user.get().id());
        return tokens.issue(user.get());
    }

    private boolean isLocked(UserCredentials u) {
        return u.lockedUntil() != null && u.lockedUntil().isAfter(clock.instant());
    }
}
```

Dans `shop-infrastructure`, `DelegatingPasswordHasher` encapsule le `PasswordEncoder` de Spring Security (Argon2id, §9.2), et `JwtTokenIssuer` signe les jetons.

E.4 Vérification automatique

✓ Garde-fous ArchUnit (règle de dépendance)

```
@AnalyzeClasses(packages = "com.vvs.shop", importOptions = ImportOption.DoNotIncludeTests.class)
class ArchitectureTest {

    @ArchTest
    static final ArchRule domain_is_pure =
        noClasses().that().resideInAPackage("com.vvs.shop.domain..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "org.springframework..", "jakarta.persistence..", "tools.jackson..",
                "com.fasterxml.jackson..", "com.vvs.shop.application..",
                "com.vvs.shop.infrastructure..", "com.vvs.shop.api..");

    @ArchTest
    static final ArchRule application_does_not_know_adapters =
        noClasses().that().resideInAPackage("com.vvs.shop.application..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "com.vvs.shop.infrastructure..", "com.vvs.shop.api..", "jakarta.persistence..");

    @ArchTest
    static final ArchRule api_does_not_touch_infrastructure =
        noClasses().that().resideInAPackage("com.vvs.shop.api..")
            .should().dependOnClassesThat().resideInAPackage("com.vvs.shop.infrastructure..");

    @ArchTest
    static final ArchRule no_field_injection =
        noFields().should().beAnnotatedWith(Autowired.class);
}
```

ArchUnit propose aussi `Architectures.onionArchitecture()` pour déclarer l'ensemble des couches en une règle ; les règles explicites ci-dessus sont préférées car leurs messages d'échec sont plus lisibles pour l'équipe.

> **Décision à valider — Annotations Spring dans la couche application (D22)**
>
> **Recommandé** : autoriser `@Service` et `@Transactional` dans `shop-application` (pragmatique, évite une couche de câblage). Variante stricte : application sans Spring, beans déclarés dans `shop-boot` — plus pur, plus de code de configuration.

E.5 Entités : domaine et persistance séparés

- **Entité de domaine** (`shop-domain`) : Java pur, sans annotation JPA, invariants dans le constructeur / la factory (`Order.create(...)`). Pas de suffixe : `Order`.
- **Entité JPA** (`shop-infrastructure`) : suffixe `JpaEntity` (`OrderJpaEntity`), structure alignée sur les tables, jamais exposée en dehors du module. Un `OrderPersistenceMapper` convertit dans les deux sens.
- Le repository de domaine (`OrderRepository`) est une interface du domaine ; `OrderRepositoryAdapter` l'implémente en s'appuyant sur `OrderJpaRepository` (Spring Data).
- `@Enumerated(EnumType.STRING)` toujours ; `fetch = LAZY` partout ; jamais d'`EAGER` ; jamais `@Data` sur une entité JPA ; `equals/hashCode` sur l'identifiant.
- Concurrence optimiste : `@Version` (type `Long`) sur l'entité JPA ; la version est portée par l'entité de domaine pour être renvoyée à la sauvegarde. `AuditedJpaEntity` est l'équivalent de `BaseEntity` (§8.1) pour les entités JPA de l'infrastructure.
- `open-in-view` désactivé : `spring.jpa.open-in-view=false`. Le mapping vers résultats se fait dans la transaction du cas d'usage.

✓ Domaine pur + entité JPA séparée

```
// shop-domain — Java pur
public final class Order {
    private final UUID id;
    private final UUID customerId;
    private OrderStatus status;
    private final BigDecimal totalAmount;
    private final Long version;                      // null tant que la commande n'est pas persistée

    private Order(UUID id, UUID customerId, OrderStatus status, BigDecimal totalAmount, Long version)
{
        this.id = id; this.customerId = customerId; this.status = status;
        this.totalAmount = totalAmount; this.version = version;
    }

    public static Order create(UUID customerId, List<OrderLine> lines) {
        Objects.requireNonNull(customerId);
        if (lines.isEmpty()) throw new InvalidOrderException("Commande sans ligne");
        var total = lines.stream().map(OrderLine::subtotal).reduce(BigDecimal.ZERO, BigDecimal::add);
        // UUIDv7 souhaité (§8.3) : l'obtenir via un port IdGenerator implémenté dans
l'infrastructure
        return new Order(UUID.randomUUID(), customerId, OrderStatus.DRAFT, total, null);
    }

    public static Order restore(UUID id, UUID customerId, OrderStatus status, BigDecimal total, Long
version) {
        return new Order(id, customerId, status, total, version);   // reconstitution depuis la base
    }

    public UUID id() { return id; }
}

// shop-infrastructure — persistance
@Entity
@Table(name = "orders")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
class OrderJpaEntity extends AuditedJpaEntity {
    @Id private UUID id;
    @Column(name = "customer_id", nullable = false) private UUID customerId;
    @Enumerated(EnumType.STRING) @Column(nullable = false, length = 20) private OrderStatus status;
    @Column(name = "total_amount", nullable = false, precision = 19, scale = 2) private BigDecimal
totalAmount;
    @Version private Long version;   // null = nouvelle entité pour Spring Data (persist, pas merge)
}
```

E.6 Exemples : cas d'usage et controller

Exemple de référence (cas d'usage Clean Architecture)

```
package com.vvs.shop.application.order;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.vvs.shop.domain.order.Order;
import com.vvs.shop.domain.order.OrderRepository;

@Service
@Transactional
public class CreateOrderService implements CreateOrderUseCase {

    private static final Logger log = LoggerFactory.getLogger(CreateOrderService.class);

    private final OrderRepository orderRepository;   // port défini dans le domaine

    public CreateOrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    @Override
    public OrderResult execute(CreateOrderCommand command) {
        Order order = Order.create(command.customerId(), command.lines());   // le domaine calcule le
total
        Order saved = orderRepository.save(order);
        log.info("Order {} created", saved.id());
        return OrderResult.from(saved);
    }
}
```

✓ Controller mince (adaptateur entrant)

```
@RestController
@RequestMapping("/api/v1/orders")
@Validated
@Tag(name = "Orders")
public class OrderController {

    private final CreateOrderUseCase createOrder;
    private final OrderApiMapper mapper;

    public OrderController(CreateOrderUseCase createOrder, OrderApiMapper mapper) {
        this.createOrder = createOrder;
        this.mapper = mapper;
    }

    @PostMapping
    @HasPermission("orders.create")
    public ResponseEntity<OrderRDto> create(@Valid @RequestBody CreateOrderWDto dto) {
        var result = createOrder.execute(mapper.toCommand(dto));
        return ResponseEntity.created(URI.create("/api/v1/orders/" +
result.id())).body(mapper.toRDto(result));
    }
}
```